



Techleaf

Technical Documentation

YOUR PATH TO BETTER FINANCIAL CHOICES

Product Name: **Techleaf**

Version: **1.0**

Phase: **Final Documentation**

Date: **April 26, 2025**

Name	Role	Contact Info	Linkedin
Darya Ahmadova	IT Business Analyst ABB	daryahmadoval6@gmail.com	
Rashad Abbasov	Deputy Director ABB	rashad.abbasov@abb-bank.az	
Samira Tayyarli	IT Business Analyst & QA Baku Stock Exchange	tayyarlisamira@gmail.com	
Zemfira Karimova	Product Owner Central Bank	karimovazemfira@yahoo.com	





REVISION HISTORY

Version	Date	Author(s)	Status	Changes
0.1	March 28, 2025	Samira	Draft	Document structure and TOC created
0.2	March 30, 2025	Darya	Draft	Added introduction and vision/mission
0.3	April 1, 2025	Zemfira	Draft	Wrote functional and non-functional requirements
0.4	April 2, 2025	Samira	Draft	Use case navigation and system requirements added
0.5	April 4, 2025	Darya	Draft	Created system design and architecture diagram
0.6	April 6, 2025	Zemfira	In Progress	Completed API documentation and flow mapping
0.7	April 8, 2025	Samira	In Progress	Drafted development and database requirements
0.8	April 10, 2025	Darya	In Progress	Wrote server/network infrastructure section
0.9	April 14, 2025	Zemfira	In Review	Finalized formatting, version table, and glossary
1.0	April 19, 2025	Zemfira	Final	Document finalized for defense submission





Table of Contents

1. Introduction	1
2. Design	2
3. Narrative Use Case	3
4. API Documentation	6
5. Functional Requirements	26
6. Non-Functional Requirements	41
7. System Requirements	55
8. Development Requirements	61
9. Database Requirements	65
10. Server and Network Infrastructure	69
11. Minimum Requirements	72
12. High-Level Architecture Diagram	75
13. Jira- MVP User Stories - Acceptance Criteria	76





1. Introduction

VISION:

Techleaf aims to shape the future of financial decision-making by connecting over **10,000 users** with tailored financial solutions, forming strategic partnerships with **15+ institutions**, achieving **80% AI match accuracy**, and securing **~60,000 in early-stage funding** to drive innovation and scale.

MISSION:

Techleaf delivers a personalized and data-driven experience that empowers individuals to discover, compare, and connect with the most relevant financial products—leveraging AI technology, real-time data, and trusted partnerships to make smarter financial choices accessible to all.

The Product

Techleaf is a digital platform that helps users discover, compare, and choose the best financial products — including loans, cards, deposits, and insurance

The Leadership

Led by Team 3 at ABB Tech Academy, the project brings together skilled business analysts and developers focused on building practical, user-centric fintech solutions.

The Overall Industry

The financial services market in Azerbaijan is growing, but product discovery remains manual and inefficient. Techleaf addresses this gap by offering a centralized, data-driven solution tailored to local users.

The Competitors

Major players include Hangikredi.com, Banks.az and individual bank platforms. Techleaf's competitive edge lies in its AI matching, real-time data scraping, and gamified user experience — all without relying on API integrations.

The Financial Status

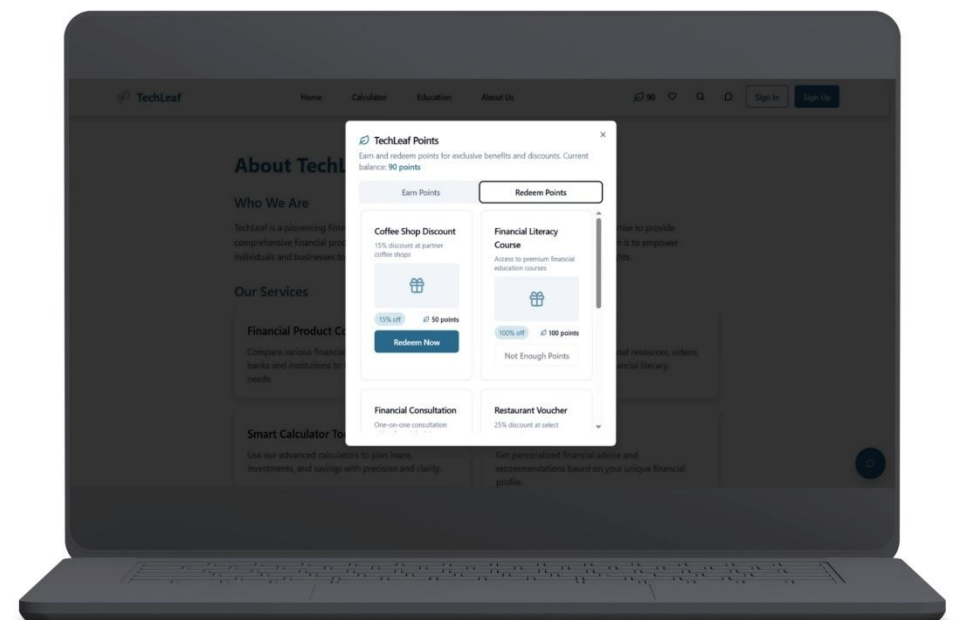
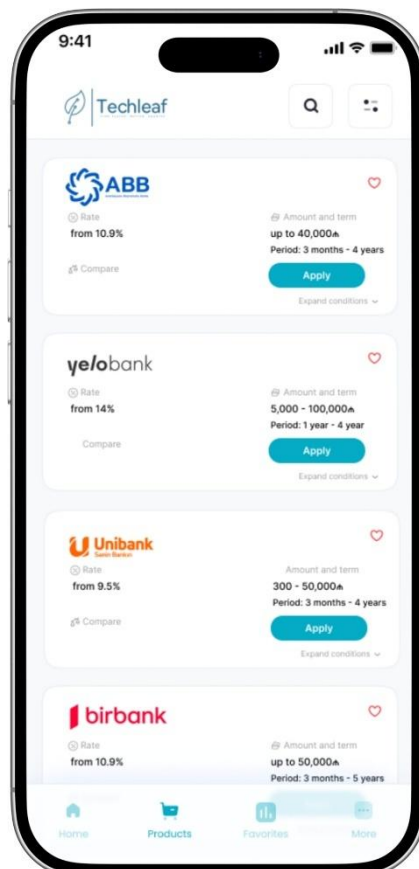
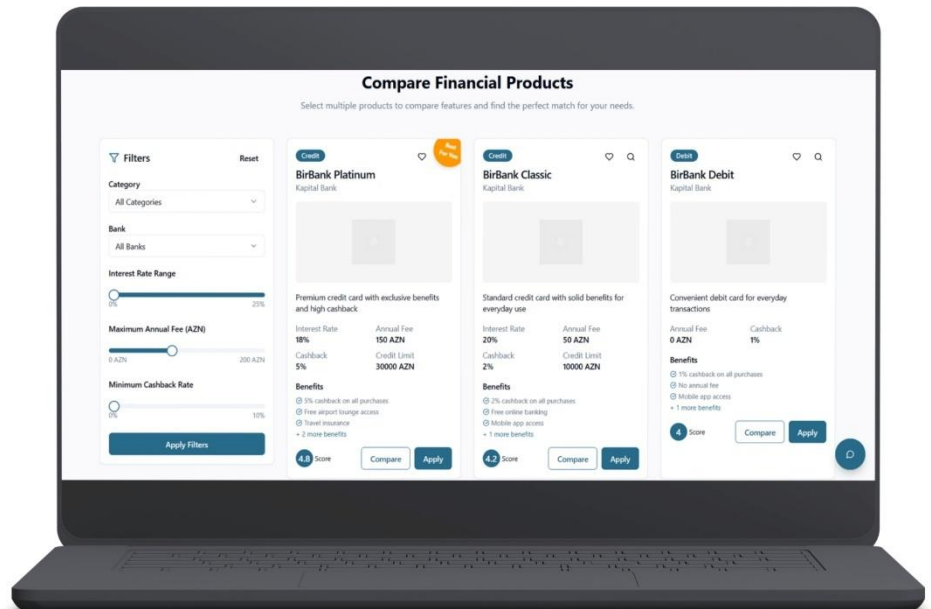
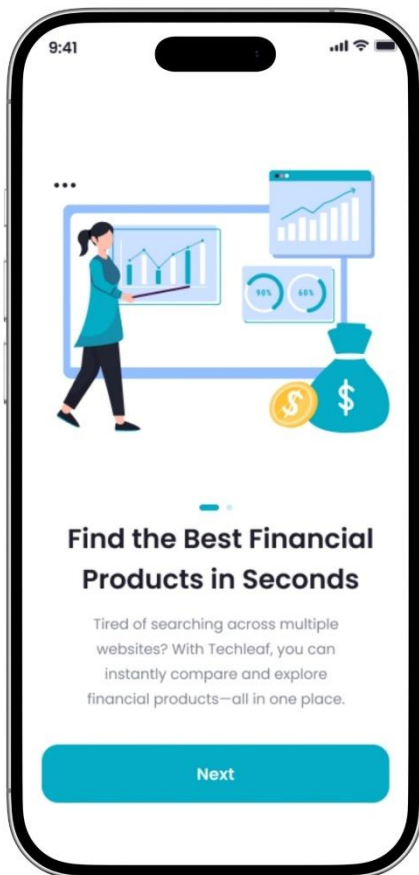
Techleaf is currently at the MVP stage and operates with a lean development model. The team seeks initial funding and strategic partnerships to support platform growth.

Future Plans

By late 2025, Techleaf aims to reach 10,000+ users and integrate with 15+ financial institutions. Mobile app development and regional expansion are part of the roadmap.



2. Design





3. Narrative Use Case

Use Case ID:	UC-APL-001		
Created By:	Techleaf Development Team	Last Updated By:	Zemfira Karimova
Date Created:	17.02.2025	Date Last Updated:	17.02.2025

Use case	User searches and applies filters to find a suitable financial product
Priority	High
Description	This use case describes how a user searches and compares various financial products (loans, deposits, bank cards, P2P lending, and insurance offers) available on the TechLeaf platform using filters in order to find the most relevant and affordable financial solution.
Actor	User (Customer), TechLeaf Platform, Banks & Credit Institutions, P2P Credit Providers, Insurance Companies
Precondition	The user must have a TechLeaf account and be logged into the system.
	"Active offers from various banks, insurance, and credit institutions must be available on the platform."
	Integrations with external systems must be functioning properly.
	Search and filter components must be in working condition.
Trigger	The user wants to find and compare the most suitable and affordable financial product that meets their needs — including loans, deposits, bank cards, or P2P credit options.
Main scenario	The user logs into the TechLeaf platform. On the search panel, the user selects the type of financial product (loan, deposit, card, insurance, or P2P loan). The user sets filters based on specific parameters (e.g., interest rate, loan term, minimum deposit amount, insurance coverage amount, etc.). The system displays the most relevant and advantageous offers according to the selected criteria. The user selects the financial product and reviews the application or offer details. If the offer requires redirection, the user is taken to the corresponding bank's or P2P lender's page.
Expected result	The user compares multiple products. With the help of filters, the user finds the most relevant and affordable product. The AI engine analyzes the user's preferences and offers the best-matching recommendations. The user's selected product(s) and terms are recorded under their profile.
Exceptions	Selected filters and parameters must be correctly applied. Product data must be accurate and up-to-date. TechLeaf's analytics system should record user activity for future suggestions.
Postcondition	The user compares the selected financial product. The user's selection is recorded in Techleaf's analytics system and used for future recommendations.
Future Development	Smart AI Recommendations – AI generates suggestions based on the user's preferences and previous selections. Premium Filter Options – Filter suggestions based on user login behavior. Premium Subscription Benefits – Special offers and discounts for premium users





Use Case ID:	UC-APL-002		
Created By:	Techleaf Development Team	Last Updated By:	Samira Tayyarli
Date Created:	20.02.2025	Date Last Updated:	20.02.2025

Use case	User watches platform-provided videos to collect points and redeem special rewards.
Priority	Medium
Description	This use case describes how the user collects points by watching specific videos on the TechLeaf platform. Accumulated points can then be exchanged for promo codes, subscriptions, e-gift cards, discounts on financial services, and exclusive rewards.
Actor	User (Customer), TechLeaf Platform, Partner Companies (reward and service providers).
Precondition	The user must have a TechLeaf account and be logged into the system.
	Video watching and point collection features must be active on the platform.
	The reward and service catalog must be available and accessible.
	The user must log in daily (for daily rewards, if applicable).
Trigger	The user wants to collect points and redeem them for digital or early-access rewards by watching a video.
Main scenario	The user logs into the TechLeaf platform. The user navigates to the "Collect Points" section. The user selects one of the available videos and starts watching it. Upon completing the video, a specific number of points are automatically added to the user's balance. The user views their point balance and browses the reward & service catalog. The user selects a reward according to the number of points collected. The selected reward is sent to the user via email or phone (e.g., promo code or subscription link).
Expected result	The user collects points by watching videos. The user redeems collected points for promo codes, subscriptions, or other digital rewards. The platform updates the user's history and shows personalized video and reward suggestions.
Exceptions	The video is not fully watched due to poor internet connection, and points are not added. Due to a technical issue, points are not credited to the user's balance. Certain rewards are unavailable in the catalog, preventing the user from completing their selection.
Postcondition	The user redeems digital rewards and services using collected points. The platform analyzes user watch and point activity to deliver personalized recommendations.
Future Development	Loyalty Program: Additional bonuses and exclusive offers for users who watch videos regularly. Exclusive Video Categories: Certain videos and rewards are accessible only to premium users.





Use Case ID:	UC-APL-003		
Created By:	Techleaf Development Team	Last Updated By:	Samira Tayyarli
Date Created:	17.02.2025	Date Last Updated:	24.02.2025

Use case	Discovering and comparing the most suitable financial product through AI-powered recommendations
Priority	High
Description	This use case describes how a user leverages the AI recommendation system on the TechLeaf platform to discover and compare personalized financial products such as loans, deposits, bank cards, and insurance options.
Actor	User (Customer), TechLeaf AI, Banks & Credit Institutions, P2P Lending Providers, Insurance Providers
Precondition	User (Customer), TechLeaf AI, Banks & Credit Institutions, P2P Lending Providers, Insurance Providers
	The AI recommendation engine must be operational and able to process data in real time.
	Offer data from banks and other financial institutions must be up to date.
Trigger	The user wants to discover the most suitable financial product based on personalized AI recommendations, without manually using search or filters.
Main scenario	The user logs into the TechLeaf platform. The AI engine analyzes the user's profile, past search behavior, and platform activity to understand preferences. The AI model asks follow-up questions if needed (e.g., goals, expenses, loan preferences). Based on responses, the AI recommends the most relevant and suitable financial products. The platform presents the user with personalized loan, deposit, card, and insurance options. The user compares the AI-recommended products. The user selects a preferred product and is redirected to the respective bank or credit provider's application page.
Expected result	The user quickly finds the most relevant financial product without manual search or filtering. The AI engine provides highly personalized and accurate recommendations. The user enjoys a smooth and seamless experience. The platform learns from user behavior to improve future recommendation accuracy.
Exceptions	The AI engine cannot generate recommendations due to insufficient financial data in the user profile. A technical issue occurs in the AI recommendation system. No suitable offers are found because data from banks and financial institutions is not up to date.
Postcondition	The user finds and compares a financial product tailored to their individual needs. The selected product is logged in Techleaf's AI system for future recommendation purposes.
Future Development	The AI model will analyze the user's financial behavior in greater depth to deliver even smarter recommendations. Integration of voice assistants and chatbots will allow users to receive answers from the AI system interactively. Premium users will have access to dedicated AI support and personalized financial advisory services.





4. API Documentation

Contents

4.1.	API Endpoints Overview.....	6
4.2.	Authentication & Authorization.....	7
4.3.	Login Endpoint.....	8
4.4.	Logout Endpoint.....	8
4.5.	Response Structure & Status Codes.....	8
4.6.	Error Response Format (Standard).....	9
4.7.	Rate Limiting.....	10
4.8.	User Management API.....	10
4.9.	Product Request API.....	14
4.10.	Product Listing & Details API.....	16
4.11.	Offer Management API.....	17
4.12.	Consent Management.....	19
4.13.	Review & Rating API.....	20
4.14.	Product & Request Insights API.....	22
4.15.	Error Codes & Handling.....	23
4.16.	API Versioning Policy.....	24

4.1. API Endpoints Overview

All Techleaf API endpoints follow RESTful design principles and use predictable, resource-oriented URLs. The API supports JSON for both request and response bodies and is secured using Bearer Token authentication.

Base URL (Production): <https://api.techleaf.az/v1/>

Base URL (Test): <https://sandbox.techleaf.az/v1/>





Endpoint	Method(s)	Description
<code>/api/users</code>	GET, POST	Retrieve all users or register a new one
<code>/api/users/{id}</code>	GET, PUT, DELETE	Fetch, update, or delete a specific user
<code>/api/auth/login</code>	POST	Authenticate user and return access token
<code>/api/consent</code>	POST	Collect user consent for data processing
<code>/api/consent/{id}</code>	DELETE	Withdraw user consent and delete data
<code>/api/auth/refresh-token</code>	POST	Refresh the session token
<code>/api/payment/transaction</code>	POST	Process secure payments (PCI-DSS compliance)
<code>/api/users/{id}/delete</code>	DELETE	Endpoint for user to delete their account, in line with GDPR
<code>/api/auth/logout</code>	POST	Logout and invalidate session token
<code>/api/products</code>	GET	List all financial products (loans, cards, deposits, insurance)
<code>/api/products/{id}</code>	GET	Retrieve detailed information for a single product
<code>/api/products/categories</code>	GET	Retrieve available product categories
<code>/api/requests</code>	GET, POST	Submit or view financial product search requests
<code>/api/requests/{id}</code>	GET, PUT, DELETE	Manage a specific request (view, update, cancel)
<code>/api/offers</code>	POST	Submit a new offer for a user request
<code>/api/offers/{id}</code>	GET, PUT	Retrieve or update a specific offer
<code>/api/offers/user/{id}</code>	GET	View all offers made for a specific user
<code>/api/reviews</code>	GET, POST	Submit or view product or institution reviews
<code>/api/reviews/{id}</code>	GET, PUT, DELETE	Manage individual review records
<code>/api/analytics/products</code>	GET	Fetch analytics on product engagement and interest trends
<code>/api/analytics/requests</code>	GET	View aggregated user request statistics
<code>/api/admin/users</code>	GET	Admin view of all platform users
<code>/api/admin/offers</code>	GET	Admin view of all submitted offers across the system
<code>/api/admin/logs</code>	GET	View audit logs of system-level actions (restricted access)

4.2. Authentication & Authorization

The Techleaf API uses **Bearer Token authentication** based on OAuth 2.0 principles. All secure endpoints require a valid access token passed in the Authorization header of the HTTP request.

Authentication Flow	User submits login credentials via <code>/api/auth/login</code> System returns a JWT access token Token is passed in the Authorization header of all subsequent requests Token expires after a predefined duration (e.g., 2 hours) Logout or session termination happens via <code>/api/auth/logout</code>
---------------------	---





4.3. Login Endpoint

```

1 Request Example
2
3 POST /api/auth/login HTTP/1.1
4 Host: api.techleaf.az
5 Content-Type: application/json
6
7 {
8   "email": "user@example.com",
9   "password": "securePassword123"
10 }
11 Response Example
12 {
13   "access_token": "eyJhbGciOiJIUzI1NiIsInR...\"",
14   "token_type": "Bearer",
15   "expires_in": 7200
16 }
17

```

4.4. Logout Endpoint

```

1 POST /api/auth/logout
2 Request:
3 POST /api/auth/logout
4 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
5 Response:
6 {
7   "message": "Logout successful."
8 }

```

Security Notes

- All tokens are signed using **HMAC SHA-256**
- Tokens must be stored securely on the client side (preferably in memory or secure storage)
- Never expose tokens in URLs or logs
- Rate limits apply to login endpoint to prevent brute-force attacks

Authentication Parameters Table

Parameter	Type	Required	Description
email	string	Yes	Registered user's email address
password	string	Yes	User's account password

```

1 Refresh Token Endpoint: POST /api/auth/refresh-token
2 Example Request:
3 POST /api/auth/refresh-token
4 {
5   "refresh_token": "<refresh_token>"
6 }
7 Example Response:
8 {
9   "status": "success",
10  "access_token": "<new_access_token>",
11  "expires_in": 7200
12 }
13

```

4.5. Response Structure & Status Codes

All responses from the Techleaf API follow a consistent JSON structure and standard HTTP status codes. This helps developers reliably handle both successful and error responses across all endpoints.

Successful Response Format (Standard)





For most GET, POST, or PUT operations, a successful response returns:

```

1 {
2   "status": "success",
3   "data": {
4     // response-specific payload
5   }
6 }
7 For list endpoints (GET /api/products, etc.):
8 {
9   "status": "success",
10  "data": [
11    {
12      "id": 101,
13      "name": "Loan Product A",
14      "interest_rate": 12.5,
15      "term_months": 24
16    },
17    ...
18  ]
19 }
```

4.6. Error Response Format (Standard)

```

1 {
2   "status": "error",
3   "error_code": "AUTH_FAILED",
4   "message": "Invalid email or password."
5 }
```

Status Code	Meaning	When It Occurs
200 OK	Request succeeded	Valid GET, PUT, DELETE operations
201 Created	Resource successfully created	After POST (e.g., user, request, review)
204 No Content	Successful, but no response body	On success without data (e.g., successful logout)
400 Bad Request	Missing or invalid parameters	Malformed JSON or input
401 Unauthorized	Authentication failed or token missing	Missing/invalid Bearer token
403 Forbidden	Authenticated but lacks permission	Trying to access protected admin route
404 Not Found	Resource does not exist	Invalid ID or endpoint
409 Conflict	Duplicate entry or business logic conflict	e.g., trying to register with an existing email
422 Unprocessable Entity	Valid JSON but semantic error	e.g., product does not match institution rules
500 Internal Server Error	Unexpected server issue	Contact support with request ID (if present)

Error Codes Reference

Error Code	Description
AUTH_FAILED	Invalid login credentials
TOKEN_EXPIRED	Access token is no longer valid





RESOURCE_NOT_FOUND	Requested entity (user, product) not found
VALIDATION_ERROR	Input failed validation rules
DUPLICATE_ENTRY	Entity already exists
ACCESS_DENIED	User is not allowed to access this resource
INTERNAL_ERROR	Unexpected failure — contact support

4.7. Rate Limiting

Techleaf applies rate limits to all API endpoints to ensure platform stability, prevent abuse, and maintain consistent performance for all partners and users.

Rate limiting is enforced on a **per-token** and **per-IP** basis, depending on the endpoint type and user role (standard user, institution, admin).

Rate limiting is implemented to ensure the stability and performance of the platform. The following limits apply for the corresponding endpoints.

Example Rate Limiting Information:

```
{
  "X-RateLimit-Limit": 100,
  "X-RateLimit-Remaining": 50,
  "X-RateLimit-Reset": 1714158703
}
```

Category	Limit	Time Window	Applies To
Authenticated Users	100 requests	per minute	General user access (search, request)
Partner Institutions	500 requests	per minute	Offer submission, analytics
Public Endpoints	50 requests	per minute	Unauthenticated endpoints (/login)
Admin Endpoints	200 requests	per minute	Admin dashboard APIs
Authentication (Login)	10 attempts	per 5 minutes	Brute-force protection

If limits are exceeded, a 429 Too Many Requests response will be returned.

Example 429 Error Response

```
{
  "status": "error",
  "error_code": "RATE_LIMIT_EXCEEDED",
  "message": "Too many requests. Please try again in 60 seconds."
}
```

4.8. User Management API

This endpoint group allows clients to register new users, retrieve user information, update user profiles, and delete accounts (admin use only). These endpoints are used by both general users and system administrators, depending on access level.





Description:

Returns a list of all registered users. Restricted to admin-level accounts.

Parameter	Type	In	Required	Description
Authorization	string	header	Yes	Bearer access token (admin)

```

3 Endpoint: GET /api/users
4 Example Request:
5 GET /api/users HTTP/1.1
6 Host: api.techleaf.az
7 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
8 Content-Type: application/json
9 Example Response:
10 {
11   "status": "success",
12   "data": [
13     {
14       "id": 1,
15       "full_name": "Elvin Mammadov",
16       "email": "elvin@example.com",
17       "created_at": "2025-03-28T14:02:33Z"
18     },
19     {
20       "id": 2,
21       "full_name": "Aynur Aliyeva",
22       "email": "aynur@example.com",
23       "created_at": "2025-03-30T09:44:21Z"
24     }
25   ]

```

```

1 Endpoint: PUT /api/users/{id}
2 Description:
3 Update the profile information of a user.
4 Users can only update their own profiles.
5 Example Request:
6 PUT /api/users/45
7 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
8
9 {
10   "phone": "+994552223344",
11   "full_name": "Leyla A. Ismayilova"
12 }

```

```

1 Endpoint: POST /api/users
2 Description:
3 Registers a new user account.
4 Example Request:
5 POST /api/users
6 Content-Type: application/json
7
8 {
9   "full_name": "Leyla Ismayilova",
10  "email": "leyla@example.com",
11  "password": "SecurePass123",
12  "phone": "+994501234567"
13 }
14 Example Response:
15 {
16   "status": "success",
17   "data": {
18     "id": 45,
19     "full_name": "Leyla Ismayilova",
20     "email": "leyla@example.com"
21   }
22 }

```

Field	Type	Required	Description
full_name	string	Yes	Full name of the new user
email	string	Yes	Email address (must be unique)
password	string	Yes	Strong password (min 8 characters)
phone	string	Optional	Contact number



```

1 Endpoint: GET /api/users/{id}
2 Description:
3 Fetch a specific user by ID.
4 Example Request:
5 GET /api/users/45
6 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
7 Example Response:
8 {
9   "status": "success",
10  "data": {
11    "id": 45,
12    "full_name": "Leyla Ismayilova",
13    "email": "leyla@example.com",
14    "phone": "+994501234567",
15    "created_at": "2025-04-12T10:10:10Z"
16  }
17 }

```

```

1 Endpoint: DELETE /api/users/{id}
2 Description:
3 Deletes a user account.
4 Only available to platform admins.
5 Example Response:
6 {
7   "status": "success",
8   "message": "User deleted successfully."
9 }

```

DELETE /api/users/{id} for
account deletion

In compliance with GDPR, users
have the right to delete their
accounts and withdraw consent.
The following endpoint allows

users to delete their account and all associated data

```

1 • Example Request:
2 DELETE /api/users/12345
3 {
4   "Authorization": "Bearer <access_token>"
5 }
6 • Example Response:
7 {
8   "status": "success",
9   "message": "User account deleted successfully."
10 }

```



The authentication endpoints handle secure user login, logout, and token management. All protected endpoints on the Techleaf platform require an access token obtained from this flow.

```

1 Endpoint: POST /api/auth/login
2 Description:
3 Authenticates a user and returns an access token (JWT) for use in protected endpoints.
4 Example Request:
5 POST /api/auth/login
6 Content-Type: application/json
7
8 {
9   "email": "leyla@example.com",
10  "password": "SecurePass123"
11 }
12 Example Response:
13 {
14   "access_token": "eyJhbGciOiJIUzI1NiIsInR...",
15   "token_type": "Bearer",
16   "expires_in": 7200
17 }
```

Field	Type	Required	Description
email	string	✓ Yes	The registered email
password	string	✓ Yes	User's password (min 8 chars)

```

1 Endpoint: POST /api/auth/logout
2 Description:
3 Logs the user out by invalidating the current access token.
4 Example Request:
5 POST /api/auth/logout
6 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
7 Example Response:
8 {
9   "status": "success",
10  "message": "Logout successful."
11 }
```

Invalid Login (Error Response)

```

{
  "status": "error",
  "error_code": "AUTH_FAILED",
  "message": "Invalid email or password."
}
```





4.9. Product Request API

This endpoint group allows users to submit requests for financial products such as loans, cards, deposits, or insurance. The system matches these requests with suitable offers from financial institutions.

```

1 Endpoint: POST /api/requests
2 Description:
3 Submit a new request for a financial product based on user preferences and profile.
4 Example Request:
5 POST /api/requests
6 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
7 Content-Type: application/json
8
9 {
10   "product_type": "loan",
11   "amount": 5000,
12   "term_months": 24,
13   "monthly_income": 1200,
14   "employed": true
15 }
16 Example Response:
17 {
18   "status": "success",
19   "data": {
20     "request_id": 78123,
21     "status": "pending",
22     "submitted_at": "2025-04-26T12:34:22Z"
23   }

```

Field	Type	Required	Description
product_type	string	✓ Yes	One of: loan, card, deposit, insurance
amount	number	✓ Yes	Desired amount (AZN)
term_months	number	✓ Yes	Requested term in months
monthly_income	number	✓ Yes	User's declared monthly income
employed	boolean	✓ Yes	Employment status

Endpoint: GET /api/requests

Description:

Returns a list of all requests submitted by the authenticated user.





```

1 Example Request:
2 GET /api/requests
3 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
4 Example Response:
5 {
6   "status": "success",
7   "data": [
8     {
9       "request_id": 78123,
10      "product_type": "loan",
11      "amount": 5000,
12      "status": "pending",
13      "created_at": "2025-04-26T12:34:22Z"
14    },
15    {
16      "request_id": 78124,
17      "product_type": "card",
18      "amount": 0,
19      "status": "responded",
20      "created_at": "2025-04-20T10:00:00Z"
21    }
22  ]
23 }

```

Endpoint: GET /api/requests/{id}

Description:

Retrieve the details of a specific request by its ID.

Endpoint: PUT /api/requests/{id}

Description:

Update request information before it's processed by a partner.

Endpoint: DELETE /api/requests/{id}

Description:

Cancel or remove a previously submitted request.

Error Cases

- 400 Bad Request: Missing required fields
- 404 Not Found: Invalid request ID
- 403 Forbidden: Trying to modify another user's request





4.10. Product Listing & Details API

This group of endpoints allows users to fetch a list of available financial products (e.g., loans, cards, deposits, insurance) along with details like interest rates, eligibility, and terms. These products are sourced via web scraping and optionally curated by institutions via the Admin Panel.

```

1 Example Request:
2 GET /api/products?type=loan
3 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
4 Example Response:
5 {
6   "status": "success",
7   "data": [
8     {
9       "id": 1001,
10      "product_type": "loan",
11      "name": "Personal Loan - Salary Card",
12      "interest_rate": 11.5,
13      "term_range_months": "6-36",
14      "max_amount": 10000,
15      "min_income": 400,
16      "institution": "Bank ABC"
17    },
18    {
19      "id": 1002,
20      "product_type": "loan",
21      "name": "Express Loan",
22      "interest_rate": 13.0,
23      "term_range_months": "3-24",
24      "max_amount": 7000,
25      "min_income": 300,
26      "institution": "BOKT FastCash"
27    }
28  ]
29 }
```

Endpoint: GET /api/products

Description:

Returns a list of all financial products currently available on Techleaf. Supports optional filtering by product type.

Parameter	Type	Required	Description
type	string	Optional	Filter by loan, card, deposit, insurance
institution	string	Optional	Filter by bank or BOKT name

Endpoint: GET /api/products/{id}

Description:

Returns detailed information about a specific financial product.



```

1 Example Request:
2 GET /api/products/1001
3 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
4 Example Response:
5 {
6   "status": "success",
7   "data": {
8     "id": 1001,
9     "product_type": "loan",
10    "name": "Personal Loan - Salary Card",
11    "description": "Fast-track personal loan with reduced interest for salary card holders.",
12    "interest_rate": 11.5,
13    "term_range_months": "6-36",
14    "min_income": 400,
15    "max_amount": 10000,
16    "institution": "Bank ABC",
17    "source_url": "https://bankabc.az/loans/personal"
18  }
19 }

```

Endpoint: GET /api/products/categories

Description:

Returns a list of supported product categories to populate filters on the frontend.

Example Response:

```

{
  "status": "success",
  "data": ["loan", "card", "deposit", "insurance"]
}

```

4.11. Offer Management API

This endpoint group allows **financial institutions** to respond to user requests with personalized offers. Offers are sent via the Admin Panel or API and are later matched and displayed to users via the mobile/web interface.

POST /api/payment/transaction

All payment data is securely tokenized and complies with **PCI-DSS** standards to ensure that sensitive financial information is encrypted both during transmission and storage

```

1 o Example Request:
2 POST /api/payment/transaction
3 {
4   "user_id": "12345",
5   "amount": 5000,
6   "payment_method": "credit_card",
7   "card_token": "<encrypted_card_token>"
8 }
9 o Example Response:
10 {
11   "status": "success",
12   "transaction_id": "txn12345",
13   "message": "Payment processed successfully."
14 }

```





Endpoint: POST /api/offers

Description:

```

1 Example Request:
2 POST /api/offers
3 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
4 Content-Type: application/json
5
6 {
7   "request_id": 78123,
8   "institution_id": "bank_abc_001",
9   "product_id": 1001,
10  "interest_rate": 11.3,
11  "amount": 5000,
12  "term_months": 24,
13  "custom_message": "Pre-approved for salary card holders"
14 }
```

Submit a product offer in response to a

```

1 Example Response:
2 {
3   "status": "success",
4   "data": {
5     "offer_id": 90762,
6     "status": "submitted"
7   }
8 }
```

specific user request. Only accessible

to authorized partner institutions.

Field	Type	Required	Description
request_id	integer	✓ Yes	ID of the user request being responded to
institution_id	string	✓ Yes	Registered ID of the financial institution
product_id	integer	✓ Yes	ID of the offered product
interest_rate	float	✓ Yes	Custom rate offered to the user
amount	float	✓ Yes	Offered amount (AZN)
term_months	integer	✓ Yes	Loan/card/deposit duration in months
custom_message	string	✗ Optional	Free-text note from the institution

Endpoint: GET /api/offers/{id}

Description:

Returns details of a specific offer.

Example Request:

```

GET /api/offers/90762
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Example Response:

```

{
  "status": "success",
  "data": {
    "offer_id": 90762,
```





```
{
  "request_id": 78123,
  "institution": "Bank ABC",
  "product": "Personal Loan - Salary Card",
  "amount": 5000,
  "interest_rate": 11.3,
  "term_months": 24,
  "status": "submitted"
}
```

Endpoint: GET /api/offers/user/{user_id}

Description:

Returns all offers made to a specific user, sorted by relevance or submission time.

Endpoint: PUT /api/offers/{id}

Description:

Allows institutions to update a submitted offer before the user responds. Usually limited to status changes or note additions.

4.12. Consent Management

- **"POST /api/consent"**: Collects user consent for data processing, as required by GDPR.
- **"DELETE /api/consent/{id}"**: Allows users to withdraw consent and delete their personal data.

Example Request:

POST /api/consent

```
{
  "user_id": "12345",
  "data_purpose": "Personalized loan offers",
  "consent_given": true
}
```

- **Example Response:**

```
{
  "status": "success",
  "message": "User consent successfully recorded."
}
```





4.13. Review & Rating API

The `/api/reviews` endpoint group allows users to submit feedback on products or institutions after receiving offers or using financial services. These reviews help future users make informed decisions and improve platform trustworthiness.

Endpoint: `POST /api/reviews`

Description:

Submits a review and rating for a product or financial institution. Users can only review entities they've interacted with (i.e., received an offer from or selected a product).

Example Request:

```
POST /api/reviews
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
Content-Type: application/json

{
  "institution_id": "bank_abc_001",
  "product_id": 1001,
  "rating": 4,
  "comment": "Fast processing and good interest rate. Recommended!"
}
```

Example Response:

```
{
  "status": "success",
  "data": {
    "review_id": 3021,
    "status": "submitted"
  }
}
```

Parameters

Field	Type	Required	Description
institution_id	string	✓ Yes	ID of the reviewed institution
product_id	integer	✓ Yes	ID of the reviewed product
rating	integer	✓ Yes	Score from 1 (worst) to 5 (best)
comment	string	✗ Optional	User feedback message (max 500 characters)

Endpoint: `GET /api/reviews`





Description:

Returns a paginated list of recent reviews across the platform. Supports optional filtering.

Example Request:

```
GET /api/reviews?institution=bank_abc_001&limit=5
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Example Response:

```
{
  "status": "success",
  "data": [
    {
      "review_id": 3021,
      "rating": 4,
      "comment": "Fast and smooth process",
      "product": "Personal Loan - Salary Card",
      "institution": "Bank ABC",
      "created_at": "2025-04-26T14:22:00Z"
    },
    ...
  ]
}
```

Endpoint: GET /api/reviews/{id}

Description:

Returns details of a single review by ID.

Endpoint: PUT /api/reviews/{id}

Description:

Allows the user to update their review (e.g., adjust rating or revise comment). Can only be done within a grace period (e.g., 7 days after submission).

Endpoint: DELETE /api/reviews/{id}

Description:

Enables users to delete their own review. Admins can remove abusive reviews.

Access & Rules

- Only **authenticated users** can post or modify reviews.
- Only **1 review per user per product/institution pair** is allowed.

Reviews are visible in the product comparison and detail views.





4.14. Product & Request Insights API

The `/api/analytics` endpoint group provides **anonymized, aggregated insights** into user behavior, product trends, and institutional performance. This is mainly used by **partner institutions** and **platform administrators** to evaluate demand and optimize offers.

Endpoint: `GET /api/analytics/products`

Description:

Returns data on the most popular financial products based on user request volume and views.

Example Request:

```
GET /api/analytics/products
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

Example Response:

```
{
  "status": "success",
  "data": [
    {
      "product_type": "loan",
      "total_requests": 3789,
      "average_amount": 6350,
      "top_institution": "Bank ABC"
    },
    {
      "product_type": "card",
      "total_requests": 2120,
      "average_amount": 0,
      "top_institution": "Bank XYZ"
    }
  ]
}
```

Endpoint: `GET /api/analytics/requests`

Description:

Returns insights on user-submitted requests: volume, categories, time trends, and resolution rates.

Example Response:

```
{
  "status": "success",
  "data": {
    "total_requests": 10500,
    "resolved_requests": 7280,
    "pending_requests": 2150,
  }
}
```





```

"request_by_category": {
  "loan": 6200,
  "card": 2800,
  "insurance": 1000,
  "deposit": 500
},
"daily_trend": [
  { "date": "2025-04-20", "requests": 420 },
  { "date": "2025-04-21", "requests": 510 },
  { "date": "2025-04-22", "requests": 478 }
]
}
}

```

Endpoint: GET /api/analytics/institution/{institution_id}

Description:

Returns institution-specific performance stats, such as average offer time, accepted offers, and product popularity.

Access Control

User Type	Access
Financial Institutions	Can view their own product and offer stats
Admin Users	Can view global and per-institution analytics
General Users	✗ No access to analytics endpoints

4.15. Error Codes & Handling

HTTP Status	Error Code	Meaning	When It Happens
400 Bad Request	VALIDATION_ERROR	The request format is invalid or required parameters are missing.	Missing email, invalid amount, malformed JSON
401 Unauthorized	AUTH_FAILED	Invalid credentials or token not provided.	Wrong email/password or no token during login
401 Unauthorized	TOKEN_EXPIRED	Token has expired or is invalid.	Access token has timed out or revoked
403 Forbidden	ACCESS_DENIED	Authenticated but not authorized to access the requested resource.	Trying to access admin or another user's data
404 Not Found	RESOURCE_NOT_FOUND	The specified resource does not exist.	Product, user, or request ID not found





409 Conflict	DUPLICATE_ENTRY	Resource already exists and conflicts with current operation.	Trying to create a user with an existing email
422 Unprocessable Entity	BUSINESS_RULE_FAIL	The request is valid, but violates business rules.	Loan request doesn't meet eligibility criteria
429 Too Many Requests	RATE_LIMIT_EXCEEDED	The client has sent too many requests in a short period.	Rate limit thresholds exceeded
500 Internal Server Error	INTERNAL_ERROR	An unexpected server-side error occurred.	Server malfunction, database error, or unhandled exception

4.16. API Versioning Policy

This section provides clear, concise definitions of key terms used throughout the Techleaf API and documentation. It ensures consistent understanding across developers, analysts, and business stakeholders.

Term	Definition
API (Application Programming Interface)	A set of defined rules that allow software components to communicate with each other.
Bearer Token	A security token used in the Authorization header to authenticate API requests.
Endpoint	A specific URL path that represents a resource (e.g., /api/users) in the API.
JWT (JSON Web Token)	A compact token used for securely transmitting user identity and permissions.
Request ID	A unique identifier for each API request, used for tracking and logging.
Offer	A financial product proposal sent by a partner institution in response to a user's request.
Product	A financial offering (loan, card, deposit, or insurance) listed on the platform.
Request (User Request)	A data entry submitted by a user specifying their financial product need.
Institution	A financial organization (bank, BOKT, insurance company) offering products on Techleaf.
Admin	A privileged user role that can access system-wide data and logs.
Review	A user-submitted rating and comment about a product or institution.





Term	Definition
JSON (JavaScript Object Notation)	A lightweight data format used for sending structured data in API requests/responses.
Status Code	A 3-digit number indicating the result of an HTTP request (e.g., 200, 404).
Rate Limiting	A restriction on the number of requests a client can make to an API in a given timeframe.
Pagination	A method of splitting large sets of results into multiple pages.
UUID	Universally Unique Identifier, often used for IDs or tracking references.
Webhook (<i>Future Release</i>)	A push-based mechanism where Techleaf can notify external systems when events occur.





5. Functional Requirements

5.1. Authentication

ID	Priority	Description	Related NFR ID	Acceptance Criteria
FR-AUTH-01	High	System shall allow users to authenticate via email and password using <code>/api/auth/login</code> .	NFR-SEC-01	Returns HTTP 200 and JWT token if credentials are valid
FR-AUTH-02	High	System shall return a JWT token and expiration time on successful login.	NFR-SEC-01	Token includes exp, sub, and user role claims
FR-AUTH-03	High	System shall reject login attempts with invalid credentials using HTTP 401 and a structured error.	NFR-SEC-02	Error includes AUTH_FAILED, human-readable message
FR-AUTH-04	High	System shall allow users to logout via <code>/api/auth/logout</code> , invalidating the current token.	NFR-SEC-01	Token blacklisted or removed from session DB
FR-AUTH-05	High	System shall securely hash and store passwords using bcrypt or a similar algorithm.	NFR-SEC-03	Plaintext passwords are never stored or retrievable
FR-AUTH-06	Medium	System shall support OTP (One-Time Password) verification via SMS for registration or login.	NFR-SEC-05	OTP must expire after 5 minutes and be validated before use
FR-AUTH-07	High	System shall enforce HTTPS for all authentication endpoints.	NFR-SEC-06	HTTP requests are rejected with 403
FR-AUTH-08	High	System shall throttle failed login attempts and lock account after 10 failures within 5 minutes.	NFR-SEC-04	Account locked and response includes LOCKED_OUT message
FR-AUTH-09	Medium	System shall notify users via email upon successful login from a new device or IP.	NFR-NOTIFY-01	Email contains timestamp, IP, device type
FR-AUTH-10	Low	System shall allow users to refresh expired tokens using a refresh token endpoint.	NFR-SEC-07	New access token returned if refresh token is valid
FR-AUTH-11	Medium	System shall log all login attempts (success or failure) with IP, timestamp, and user ID.	NFR-AUDIT-01	Accessible via <code>/api/admin/logs</code> for authorized roles
FR-AUTH-12	Low	System shall allow super admins to revoke user sessions remotely via admin interface or API.	NFR-SEC-08	User is logged out and future requests return 401
FR-AUTH-13	Medium	System shall support social login (Google, Apple) via OAuth 2.0 in future releases.	NFR-INT-02	User is redirected and token is validated with identity provider
FR-AUTH-14	Low	System shall allow users to reset forgotten passwords via email-based token or OTP flow.	NFR-SEC-09	Reset link/OTP is single-use and expires within 10 minutes
FR-AUTH-15	Low	System shall log token expiration and automatic logout events.	NFR-AUDIT-02	Logs viewable in admin dashboard or exported
FR-AUTH-16	Medium	System shall allow only one active session per user unless multi-device login is enabled.	NFR-SEC-10	New login replaces previous token session
FR-AUTH-17	High	System shall return a structured error response for all failed authentication attempts.	NFR-GEN-01	Format includes status, error_code, and message





FR-AUTH-18	Medium	System shall allow admin-level users to authenticate via a separate admin login interface/API.	NFR-SEC-11	Admin token contains role: admin/super_admin
FR-AUTH-19	Medium	System shall automatically expire access tokens after 2 hours by default.	NFR-SEC-01	Token must contain exp and reject after expiry
FR-AUTH-20	Low	System shall audit authentication API usage volume for rate-limiting analytics.	NFR-MON-01	Summary metrics available to devops logs

5.2. User Management

ID	Priority	Description	Related NFR ID	Acceptance Criteria
FR-USER-01	High	System shall allow users to register using email, phone number, full name, and password.	NFR-SEC-01	Registration creates a new record and sends confirmation
FR-USER-02	High	System shall validate email format and phone number format (E.164) during registration.	NFR-SEC-02	Invalid formats are rejected with detailed error
FR-USER-03	High	System shall allow users to verify their phone number via OTP before completing registration.	NFR-SEC-05	OTP expires after 5 minutes and must match to succeed
FR-USER-04	High	System shall store user profile data including email, phone, full name, birthdate, and employment.	NFR-DB-01	Data is viewable and editable via /api/users/{id}
FR-USER-05	High	System shall allow users to update their profile information excluding immutable fields (email).	NFR-SEC-06	Only editable fields allowed (e.g., phone, address)
FR-USER-06	Medium	System shall allow users to delete their account via the frontend with confirmation flow.	NFR-GEN-02	Account is soft-deleted and flagged in DB
FR-USER-07	High	System shall support user roles including standard user, partner (bank/BOKT), admin, and lender.	NFR-AUTH-01	Roles are enforced in all protected endpoints
FR-USER-08	Medium	System shall allow admin users to promote or demote user roles via the admin panel.	NFR-SEC-08	Audit log is created on every role change
FR-USER-09	High	System shall display role-specific dashboards after login (user, institution, admin).	NFR-UX-01	Correct dashboard shown based on decoded JWT role claim
FR-USER-10	Medium	System shall allow lenders (P2P role) to upload verification documents (passport, ID, etc.).	NFR-KYC-01	Uploaded files stored securely and shown in admin panel
FR-USER-11	Medium	System shall allow borrowers (P2P role) to declare financial history and loan purpose.	NFR-KYC-02	Data submitted through /api/users/{id}/p2p-profile
FR-USER-12	High	System shall display a read-only user profile summary in the admin interface.	NFR-AUDIT-01	Admin sees last login, request count, review history
FR-USER-13	Medium	System shall allow institutions to register for admin access by request form (not auto-signup).	NFR-ADMIN-01	Requests are reviewed manually and role assigned by admin
FR-USER-14	High	System shall prevent duplicate registration using the same email or phone number.	NFR-SEC-02	Conflict returns HTTP 409 and DUPLICATE_ENTRY error





FR-USER-15	High	System shall allow admin users to block or deactivate user accounts.	NFR-SEC-08	Blocked users receive 403 on all endpoints
FR-USER-16	Medium	System shall allow users to download their personal data in JSON or PDF format.	NFR-EXPORT-01	Trigger available from /profile/download
FR-USER-17	Medium	System shall notify users via email when profile information is updated.	NFR-NOTIFY-01	Email contains timestamp and changed fields
FR-USER-18	Medium	System shall allow QA/admins to impersonate users for troubleshooting with audit log.	NFR-AUDIT-03	All impersonations logged with timestamp and admin ID
FR-USER-19	Low	System shall auto-suggest institution name if user enters partner registration form.	NFR-UX-02	Search via /api/institutions/search?q=...
FR-USER-20	High	System shall ensure user data is encrypted at rest and during transmission.	NFR-SEC-03	Database fields encrypted via AES256 or equivalent

5.3. Requests

ID	Priority	Description	Acceptance Criteria
FR-REQ-01	High	System shall allow authenticated users to submit product requests via /api/requests.	Request created with status pending and assigned unique ID
FR-REQ-02	High	System shall allow request types to include loan, card, deposit, insurance.	Only supported values accepted; others return validation error
FR-REQ-03	High	System shall require mandatory fields: amount, term, income, and product type during request creation.	Request with missing fields returns HTTP 400 and error code
FR-REQ-04	High	System shall validate the business logic (e.g., max amount per income, term range).	Rules enforced dynamically per product type
FR-REQ-05	High	System shall allow users to view all their requests via /api/requests.	List sorted by created_at DESC
FR-REQ-06	Medium	System shall allow users to update or delete a request before it receives any institutional offer.	Update fails with error if offer already linked
FR-REQ-07	High	System shall notify institution partners in real time when a new relevant request is submitted.	Notification sent via Admin Panel or webhook
FR-REQ-08	High	System shall prevent duplicate submissions of the same request content within 60 seconds.	Second identical submission blocked
FR-REQ-09	Medium	System shall allow admin users to view all requests via /api/admin/requests.	Admins can filter by user, type, date, and status
FR-REQ-10	Medium	System shall display the status of each request: pending, responded, cancelled, expired.	Status is updated by system/institutions
FR-REQ-11	Medium	System shall allow users to re-submit a cancelled or expired request.	New request record is created with reference to previous
FR-REQ-12	Medium	System shall allow P2P borrowers to submit loan requests with additional P2P-specific fields.	Includes loan purpose, guarantees, and document attachments
FR-REQ-13	Medium	System shall allow users to withdraw a request if no offer has been made yet.	Status updated to cancelled and visible in dashboard
FR-REQ-14	High	System shall associate each request with a unique user ID and timestamp in database.	Ensures traceability for audit or debugging
FR-REQ-15	Medium	System shall allow filtering of request history by product type and status on frontend.	UI supports dropdown filters
FR-REQ-16	Low	System shall log all request submission and status change events in audit logs.	Log includes user ID, IP, and timestamp
FR-REQ-17	Medium	System shall allow QA or admin roles to flag suspicious requests for review.	Flags appear in Admin Panel





FR-REQ-18	Medium	System shall return request metadata in response, including match score (future AI), status, and linked offers.	Visible in <code>/api/requests/{id}</code>
FR-REQ-19	Medium	System shall allow admin to export requests as CSV or JSON via Admin Panel.	Supports date range and status filters
FR-REQ-20	Medium	System shall support request ID traceability in user-facing communications (e.g., email, SMS).	ID is visible in user inbox / notification feed

5.4. Offers

ID	Priority	Description	Acceptance Criteria
FR-OFF-01	High	System shall allow institutions to submit offers in response to valid user requests.	Offer is created and associated with request ID
FR-OFF-02	High	System shall require institutions to submit offer fields including product ID, interest, amount, and term.	Missing fields result in validation error
FR-OFF-03	High	System shall prevent institutions from responding to requests not intended for them (based on filters).	Unauthorized access returns 403
FR-OFF-04	High	System shall notify users upon receiving a new offer via in-app message and optional email.	Notification includes offer summary and institution name
FR-OFF-05	High	System shall allow users to view all received offers linked to a submitted request.	Offer cards shown in dashboard with sorting/filtering
FR-OFF-06	Medium	System shall allow users to mark one offer as accepted, closing the request.	Other offers become inactive; status = closed
FR-OFF-07	Medium	System shall allow institutions to withdraw or expire an offer before it is accepted.	Offer status changes to withdrawn or expired
FR-OFF-08	Medium	System shall prevent duplicate offers from the same institution on the same request.	HTTP 409 DUPLICATE_OFFER returned
FR-OFF-09	Medium	System shall timestamp all offer submissions and store offer version history.	Offer timeline viewable by admin
FR-OFF-10	High	System shall allow admin users to view, filter, and delete offers via Admin Panel.	Admin can filter by institution, status, request ID
FR-OFF-11	Medium	System shall allow institutions to add a custom message or condition in the offer.	Message is shown to user with offer detail
FR-OFF-12	Medium	System shall allow offers to include dynamic interest rates if supported by product config.	Displayed as floating or range
FR-OFF-13	High	System shall allow offers to be retrieved by request ID or by user ID for dashboards.	<code>/api/offers/user/{id}</code> and <code>/api/offers/request/{id}</code> work
FR-OFF-14	Medium	System shall log every offer interaction (viewed, accepted, expired) in audit logs.	Logs stored with user ID and timestamps
FR-OFF-15	High	System shall allow P2P lenders to send offers to borrower requests via dedicated endpoint.	Endpoint secured with lender role validation
FR-OFF-16	Medium	System shall allow users to compare multiple offers by interest, term, and total repayment.	Comparison logic handled on client or via <code>/api/compare</code>
FR-OFF-17	Medium	System shall send follow-up reminders to users if they haven't responded to an offer within 72 hours.	Background job checks and triggers reminders
FR-OFF-18	Medium	System shall allow institutions to tag offers with priority, approval probability, or partner campaign.	Tags used for analytics and recommendation engine
FR-OFF-19	Medium	System shall return standardized error codes for offer submission failures.	Codes like <code>INVALID_PRODUCT_ID</code> , <code>AMOUNT_TOO_HIGH</code> , etc.
FR-OFF-20	High	System shall ensure all offer data is encrypted at rest and during transmission.	AES256 encryption, HTTPS enforced
FR-OFF-21	Low	System shall allow users to export received offers in PDF or CSV format for offline review.	Trigger button from dashboard; file includes full offer data





FR-OFF-22	Medium	System shall allow institutions to see offer engagement metrics (views, time to accept).	Admin dashboard charts or API access
FR-OFF-23	Medium	System shall auto-expire offers after a configurable duration (e.g., 7 days) if no action is taken.	Status changed to expired, user notified
FR-OFF-24	Low	System shall allow institutions to attach PDF terms & conditions or rate sheets to an offer.	Attachment shown as downloadable link
FR-OFF-25	Medium	System shall allow users to filter offers by rate, term, and institution name in the UI.	UI supports dropdowns and real-time filtering

5.5. Products

ID	Priority	Description	Acceptance Criteria
FR-PROD-01	High	System shall display all active financial products via /api/products.	Product list is paginated and filterable
FR-PROD-02	High	System shall categorize products as loan, card, deposit, or insurance.	Categories appear in filters and metadata
FR-PROD-03	High	System shall allow filtering products by institution, rate, term, type, and currency.	At least 3 filters must be supported in MVP
FR-PROD-04	Medium	System shall allow sorting products by interest rate, popularity, and newest.	Sorting toggles available in UI
FR-PROD-05	High	System shall allow users to view detailed product information via /api/products/{id}.	Response includes full fields, source, and institution name
FR-PROD-06	Medium	System shall mark each product as web_scraped, submitted_by_partner, or admin_added.	Field available in API for transparency
FR-PROD-07	Medium	System shall highlight recommended or top-rated products based on engagement or rating.	recommended flag is visible in response
FR-PROD-08	Medium	System shall allow admin users to add, update, or deactivate product listings.	Form and API available under Admin Panel
FR-PROD-09	High	System shall allow scraped product entries to be updated automatically via cron scheduler.	Last updated timestamp shown in DB
FR-PROD-10	Medium	System shall display the institution name, logo, and contact info with each product.	Displayed in product card and detail page
FR-PROD-11	Medium	System shall support displaying both fixed and floating interest rates.	Fields like rate_min, rate_max rendered when applicable
FR-PROD-12	Medium	System shall allow users to save favorite products for comparison.	Favorites stored per user in DB
FR-PROD-13	Low	System shall allow users to report outdated or misleading product information.	Report triggers moderation alert to admin
FR-PROD-14	Medium	System shall allow displaying of user review summary for a product if available.	Rating average, review count visible on product card
FR-PROD-15	Medium	System shall store all product data in structured fields: interest, term, income req, fees, etc.	All fields mapped to internal schema
FR-PROD-16	High	System shall expose product categories via /api/products/categories for use in UI.	Response: ["loan", "card", "deposit", "insurance"]
FR-PROD-17	Medium	System shall allow users to search product names or institutions via keyword query.	Full-text search API supports minimum 2-character queries
FR-PROD-18	Low	System shall track and expose view count and click-through rates per product.	Field view_count updated asynchronously
FR-PROD-19	Medium	System shall allow partner institutions to request edits or takedown of incorrect product listings.	Admin receives request with product ID and reason
FR-PROD-20	Medium	System shall allow P2P products to include unique product attributes such as collateral info.	Special fields available under P2P product type
FR-PROD-21	Medium	System shall expose product metadata (scrape frequency, last updated) for admin viewing.	Viewable in Admin Panel table or export





FR-PROD-22	Low	System shall support localization of product labels and descriptions.	Fields in Azerbaijani, Russian, English
FR-PROD-23	Medium	System shall flag discontinued or inactive products and hide them from user-facing endpoints.	Status field used to control visibility
FR-PROD-24	Low	System shall allow exporting the full product list as CSV or JSON via admin endpoint.	Filterable by type, institution, updated_at
FR-PROD-25	Medium	System shall support product tags (e.g., "cashback", "low interest", "green loan").	Tags searchable and filterable

5.6. Reviews

ID	Priority	Description	Related NFR ID	Acceptance Criteria
FR-REV-01	High	System shall allow authenticated users to submit a review and rating (1–5 stars) for an institution or product.	NFR-AUTH-01	Only available if user received an offer or used product
FR-REV-02	High	System shall store review fields: user ID, product ID, rating, comment, institution ID, timestamp.	NFR-DB-01	All fields are stored in structured form
FR-REV-03	Medium	System shall limit reviews to one per user per product or institution.	NFR-SEC-01	Duplicate submission returns DUPLICATE_REVIEW error
FR-REV-04	High	System shall expose all public reviews via /api/reviews with pagination and optional filters.	NFR-UX-01	Users can filter by institution, product type, or rating
FR-REV-05	High	System shall display average rating and review count in product and institution detail pages.	NFR-GEN-01	Aggregated values updated after each new review
FR-REV-06	Medium	System shall allow users to edit their own review within 7 days of submission.	NFR-UX-02	Edit endpoint is restricted after deadline
FR-REV-07	Medium	System shall allow users to delete their own review at any time.	NFR-UX-02	Review removed from public API view
FR-REV-08	Medium	System shall allow reviews to be marked as "verified" if based on accepted offer or completed product usage.	NFR-TRUST-01	Verified badge displayed in UI
FR-REV-09	Medium	System shall allow users to flag inappropriate or fake reviews.	NFR-MOD-01	Admin receives flagged review with reason
FR-REV-10	Medium	System shall allow admin users to view, approve, hide, or delete any submitted review.	NFR-ADMIN-02	Admin actions logged with reason
FR-REV-11	Medium	System shall automatically hide reviews containing profanity or abusive language.	NFR-CONTENT-01	Uses keyword-based content moderation engine
FR-REV-12	Low	System shall send users an email notification after submitting a review to confirm receipt.	NFR-NOTIFY-01	Email includes copy of review text and timestamp
FR-REV-13	Medium	System shall expose a public review summary API for each institution: avg rating, count, verified %, etc.	NFR-ANALYTICS-01	/api/reviews/summary?institution_id=... returns JSON
FR-REV-14	Medium	System shall allow institutions to respond to reviews via a threaded comment system (admin-moderated).	NFR-COMM-02	Replies shown below reviews with institution ID
FR-REV-15	Low	System shall display review timestamps and sort reviews by newest or rating.	NFR-UX-04	Sort options available in review UI





FR-REV-16	Medium	System shall anonymize user identity in public review listings (e.g., initials or masked name).	NFR-PRIVACY-01	Only admin can view full identity
FR-REV-17	Medium	System shall restrict reviews to users with an accepted offer or completed request.	NFR-TRUST-01	Backend checks request/offer logs before accepting review
FR-REV-18	Medium	System shall allow exporting reviews for a product or institution in CSV or JSON via admin API.	NFR-EXPORT-01	Filterable by type, rating, time
FR-REV-19	Low	System shall include review analytics in the Admin Dashboard (daily count, avg rating trends).	NFR-DASHBOARD-01	Charts displayed in partner panel
FR-REV-20	Medium	System shall allow reviews to be sorted and ranked by helpfulness votes (likes/upvotes).	NFR-UX-05	Logged-in users can vote once per review

5.7. Admin Access

ID	Priority	Description	Acceptance Criteria
FR-ADMIN-01	High	System shall allow admin users to log in via /api/admin/login with separate authentication logic.	Token contains role admin or super_admin
FR-ADMIN-02	High	System shall allow admins to view and search all registered users in the system.	Search by name, email, phone, and role
FR-ADMIN-03	High	System shall allow admins to block or deactivate user accounts.	Blocked users receive 403 on API calls
FR-ADMIN-04	Medium	System shall allow admins to view all submitted product requests with filters.	Filter by user, date, type, or request status
FR-ADMIN-05	Medium	System shall allow admins to view and delete inappropriate or duplicate offers.	Deleted offers hidden from users
FR-ADMIN-06	Medium	System shall allow admins to review, approve, or hide user-submitted reviews.	All moderation actions logged with timestamp
FR-ADMIN-07	Medium	System shall allow admins to impersonate a user for debugging and support purposes.	Impersonation actions logged, visible only to super admins
FR-ADMIN-08	Medium	System shall allow admins to view platform-wide logs for authentication, requests, and offers.	/api/admin/logs includes event type, user ID, IP, and time
FR-ADMIN-09	Medium	System shall allow exporting data from the admin dashboard (users, offers, requests) in CSV/JSON.	Admin can filter and export large datasets
FR-ADMIN-10	High	System shall enforce strict role-based permissions for each admin function.	viewer, editor, super_admin roles enforced
FR-ADMIN-11	Low	System shall allow admins to receive flagged content (reviews, requests) for review.	Viewable in Admin Inbox
FR-ADMIN-12	Medium	System shall allow super admins to create, update, or remove institution partner accounts.	CRUD operations on partner accounts via UI or API
FR-ADMIN-13	Medium	System shall support viewing product submission logs from scraping, partner, or admin channels.	Source type shown in log view
FR-ADMIN-14	Medium	System shall support basic analytics in the admin panel: number of requests, offers, users per day.	Charts and tables shown on admin dashboard
FR-ADMIN-15	Medium	System shall allow configuration of business logic thresholds (e.g., max request amount).	Values editable via admin settings page
FR-ADMIN-16	Low	System shall allow admins to reset user passwords manually.	Audit log entry created for password reset
FR-ADMIN-17	Medium	System shall allow admins to toggle visibility of specific product entries (e.g., hide scraped loan).	Products hidden do not appear in /products API





FR-ADMIN-18	Medium	System shall allow admins to tag offers and requests for internal classification.	Tags stored in metadata and shown in logs
FR-ADMIN-19	Low	System shall notify admins when system-wide errors (500s, database issues) exceed a threshold.	Email or SMS alerts triggered via monitoring tool
FR-ADMIN-20	Medium	System shall provide advanced filters for reviewing offers by interest rate, institution, duration.	Multi-filter UI for offer list

5.8. Analytics

ID	Priority	Description	Acceptance Criteria
FR-ANA-01	High	System shall track the number of product views, request submissions, and offers sent per day.	Metrics stored and available for querying by admin/partners
FR-ANA-02	Medium	System shall provide daily, weekly, and monthly aggregates of requests by product type.	Charts available on admin dashboard
FR-ANA-03	Medium	System shall provide offer engagement stats to institutions (views, accept rate, time-to-accept).	/api/analytics/offers/{institution_id} provides structured data
FR-ANA-04	Medium	System shall track user growth over time, segmented by acquisition source (referral, direct, etc.).	Metrics accessible from admin analytics panel
FR-ANA-05	Medium	System shall provide a heatmap of product popularity based on user filters and interactions.	Color-coded visualization in admin panel
FR-ANA-06	Medium	System shall allow partner institutions to view top 10 most requested products per category.	Displayed in partner dashboard with optional CSV export
FR-ANA-07	Medium	System shall track reviews per institution and display rating evolution over time.	Line chart showing avg. rating per month
FR-ANA-08	Medium	System shall track failed request match attempts and reasons (e.g., no offers returned).	Breakdown of failure causes available for admin review
FR-ANA-09	Medium	System shall allow admin users to filter analytics by time range, product type, and region.	Analytics UI supports dropdown filters
FR-ANA-10	Medium	System shall store historical analytics for at least 12 months for reporting.	Data retention policy enforced
FR-ANA-11	Medium	System shall track top institutions by accepted offers and performance ratios.	Metrics displayed on leaderboard-style UI
FR-ANA-12	Low	System shall export aggregated data sets for visualization in third-party tools (Power BI, Tableau).	API or download for monthly report format
FR-ANA-13	Medium	System shall measure drop-off rates during the request form submission process.	Analytics API logs form abandonment with step
FR-ANA-14	Medium	System shall expose /api/analytics/products, /requests, and /institutions endpoints.	Public or secured endpoints return aggregate JSON
FR-ANA-15	Medium	System shall support auto-generated weekly summary reports emailed to admins and partners.	Email includes offer volume, top products, and user stats
FR-ANA-16	Medium	System shall track offer rejection reasons and visualize patterns across institutions.	Admin can see "too high interest", "low amount", etc. breakdown





FR-ANA-17	Medium	System shall track and report conversion funnel: product view → request → offer → accepted.	Conversion ratios and % shown in dashboard
FR-ANA-18	Medium	System shall allow custom metric tagging on campaigns and track their performance.	Partners can tag offers with campaign ID for analytics
FR-ANA-19	Low	System shall support API rate-limiting analytics for DevOps and load management.	Log includes endpoint, IP, and timestamp
FR-ANA-20	Low	System shall allow exporting full analytics datasets monthly in encrypted format.	Admin panel supports ZIP archive download

5.9. Moderation & Validation

ID	Priority	Description	Acceptance Criteria
FR-MOD-01	High	System shall validate all required fields (e.g., amount, term, type) before saving user requests.	Missing/invalid data rejected with HTTP 400 and error code
FR-MOD-02	High	System shall restrict input values within acceptable business ranges (e.g., loan term ≤ 60 months).	Invalid input returns BUSINESS_RULE_VIOLATION
FR-MOD-03	High	System shall reject offers from institutions if linked request is no longer active.	Returns HTTP 403 with error REQUEST_EXPIRED_OR_CLOSED
FR-MOD-04	Medium	System shall flag reviews containing profanity or abusive terms for admin approval.	Flagged reviews hidden from public feed
FR-MOD-05	Medium	System shall allow admins to review flagged reviews, requests, or offers with reason tagging.	Admin can approve, edit, or delete
FR-MOD-06	Medium	System shall prevent the same user from submitting identical request data more than once per minute.	Rate-limiting logic returns DUPLICATE_REQUEST
FR-MOD-07	Medium	System shall validate uploaded documents (PDF, JPG) for size, type, and integrity.	Max size = 5MB, invalid types return UNSUPPORTED_FILE_TYPE
FR-MOD-08	Medium	System shall allow admin to blacklist institutions from submitting offers if flagged repeatedly.	Admin panel includes block/unblock option
FR-MOD-09	Medium	System shall flag suspicious activity (e.g., >10 requests in 1 minute) and notify admin.	Email or dashboard alert triggered
FR-MOD-10	Medium	System shall validate scraped product entries for completeness and structural integrity.	Scraped items missing critical fields are excluded from UI
FR-MOD-11	Medium	System shall allow partners to report data discrepancies in scraped products.	Form submits to admin with product ID and error type
FR-MOD-12	Medium	System shall detect and suppress spam-like reviews (e.g., repeated content from same user/IP).	Spam flagged using basic NLP heuristics
FR-MOD-13	Low	System shall notify users if their submitted content (request/review) was moderated or edited.	Message includes reason and original content snapshot
FR-MOD-14	Medium	System shall block banned words and phrases from being submitted via free text inputs.	Input returns validation error on detection





FR-MOD-15	Medium	System shall log all moderation actions with admin ID, content type, and timestamp.	Logs viewable via /admin/logs or moderation dashboard
FR-MOD-16	Low	System shall allow admin to create and update custom validation rules via admin interface.	Rules stored in config DB and used in validation logic
FR-MOD-17	Medium	System shall return structured error objects for all failed validations.	Format: { "error_code": "...", "message": "...", "field": "..." }
FR-MOD-18	Medium	System shall support A/B validation rules during pilot phases (e.g., different loan ranges per segment).	Variant group stored in user session
FR-MOD-19	Low	System shall auto-block users/IPs after repeated invalid submission patterns.	Threshold-based logic blocks IP temporarily
FR-MOD-20	Medium	System shall alert admin if product scraping pipeline yields anomalies (e.g., no updates, field drift).	Monitoring logs + Slack/email alert on anomalies

5.10. Search & Filtering

ID	Priority	Description	Acceptance Criteria
FR-SEARCH-01	High	System shall allow users to search financial products by keyword (e.g., product name or institution name).	Search results must return within 1.5 seconds
FR-SEARCH-02	High	System shall allow filtering products by category (loan, deposit, card, insurance).	Filters appear on product listing page
FR-SEARCH-03	High	System shall allow filtering by interest rate range, term length, institution, and approval probability.	Dropdowns or sliders enable multi-filtering
FR-SEARCH-04	Medium	System shall allow users to sort products by popularity, lowest rate, highest rating, or newest.	Sort order affects visible results and URL state
FR-SEARCH-05	Medium	System shall allow full-text search across product descriptions, institution bios, and category tags.	/api/search?q=term endpoint returns results by relevance
FR-SEARCH-06	Medium	System shall provide auto-suggestion dropdowns as users type keywords into the search bar.	Suggestions from recent queries or tag index
FR-SEARCH-07	Medium	System shall allow users to filter their past requests by status (pending, accepted, expired).	Available under /dashboard/requests
FR-SEARCH-08	Medium	System shall allow filtering of offers by institution, interest rate, loan term, and review rating.	Filters apply instantly to visible offer list
FR-SEARCH-09	Medium	System shall persist the last used filters in session/local storage for better UX.	Returning users see same filters re-applied
FR-SEARCH-10	Low	System shall support voice-to-text input for mobile search queries.	Browser-level support for voice search
FR-SEARCH-11	Low	System shall allow admin users to filter user, request, or offer data in their dashboard.	Admin can filter by date, type, and region
FR-SEARCH-12	Low	System shall allow combining filters using AND/OR logic in advanced search mode.	e.g., "Interest rate < 10% AND institution = Bank X"
FR-SEARCH-13	Medium	System shall include a tag cloud or filter list with top trending search terms.	Data pulled from analytics index
FR-SEARCH-14	Medium	System shall highlight matched search terms in product or institution results.	Highlighted terms bolded or colored
FR-SEARCH-15	Low	System shall allow export of search result sets for admin (filtered offers, users, or products).	Button generates CSV of current filter state
FR-SEARCH-16	Medium	System shall log user search queries and filters for analytics and future ranking improvements.	Stored securely for up to 12 months





FR-SEARCH-17	Low	System shall allow partner institutions to search all submitted user requests (limited by filters).	/api/partner/requests with role-based access control
FR-SEARCH-18	Medium	System shall support real-time filtering without page reload via AJAX or GraphQL.	User does not experience refresh when applying filters
FR-SEARCH-19	Medium	System shall support filtering by review sentiment once integrated with NLP analytics.	Filter: "Positive / Neutral / Negative" sentiment
FR-SEARCH-20	Medium	System shall allow filtering of P2P-only products or requests by type of lender/borrower.	Shown only if P2P is enabled for the platform

5.11. Rate Limiting

ID	Priority	Description	Acceptance Criteria
FR-RATE-01	High	System shall limit unauthenticated requests to 100 requests per IP per hour.	Exceeding limit returns HTTP 429 and retry header
FR-RATE-02	High	System shall limit authenticated users to 1000 API requests per day.	Token identified; limit tracked in Redis
FR-RATE-03	High	System shall include the following rate-limit headers in responses: X-RateLimit-Limit, Remaining, Reset.	Headers appear in every applicable response
FR-RATE-04	Medium	System shall allow admin override of limits for testing or QA environments.	Configurable via environment variable
FR-RATE-05	Medium	System shall implement per-endpoint throttling rules for sensitive operations (e.g., login, search).	Login: 10/min, Search: 20/min per IP
FR-RATE-06	Medium	System shall store rate-limit usage in-memory for fast access (Redis or equivalent).	Usage counters persist for TTL (time-to-live)
FR-RATE-07	Medium	System shall reset all user-specific rate limits daily at midnight UTC.	Verified by logs; reset does not affect global counters
FR-RATE-08	Medium	System shall apply stricter rate limits on /api/auth endpoints to prevent brute-force login.	5 attempts per minute allowed; then cooldown applied
FR-RATE-09	Medium	System shall block traffic from IP addresses that exceed rate limits repeatedly within 24 hours.	Blacklisted IPs stored temporarily in firewall logic
FR-RATE-10	Low	System shall notify admin if more than 5 IPs hit global limits within 5 minutes.	Alert sent via email or Slack webhook
FR-RATE-11	Medium	System shall allow rate limits to be configured per institution partner or role.	Partners may request higher limits via admin approval
FR-RATE-12	Medium	System shall log all rate-limiting events with timestamp, IP, endpoint, and user ID.	Logs available for admin or security review
FR-RATE-13	Medium	System shall return an error response with error_code: RATE_LIMIT_EXCEEDED.	Consistent with other platform-wide error formats
FR-RATE-14	Medium	System shall support burst tolerance (e.g., 20 requests instantly but not sustained beyond threshold).	Token bucket algorithm or leaky bucket configured
FR-RATE-15	Low	System shall allow QA engineers to bypass limits on staging environment using special API key.	Identified by header x-debug-key
FR-RATE-16	Medium	System shall rate-limit SMS/email notifications to 3 per minute per user to prevent abuse.	Validation logic integrated into message service
FR-RATE-17	Medium	System shall allow admin users to view rate-limit logs from the admin panel.	View sorted by IP, endpoint, hit count
FR-RATE-18	Medium	System shall display a clear rate-limit error message to end-users on frontend (e.g., "Try again later").	Localized error shown in toast/modal
FR-RATE-19	Medium	System shall document rate limits per endpoint in the API documentation.	Markdown table with limits for dev reference
FR-RATE-20	Low	System shall provide a GET /api/ratelimit/status endpoint for authenticated users.	Returns remaining requests and reset timestamp





5.12. Logging & Auditing

ID	Priority	Description	Related NFR ID	Acceptance Criteria
FR-LOG-01	High	System shall log all successful and failed login attempts with user ID, IP, and timestamp.	NFR-AUDIT-01	Stored in /admin/logs/auth
FR-LOG-02	High	System shall log all user-submitted requests with full payload (excluding PII).	NFR-AUDIT-02	Entry includes product type, amount, request ID
FR-LOG-03	High	System shall log all offer submissions, updates, and cancellations.	NFR-AUDIT-02	Includes institution ID, offer ID, and reason if deleted
FR-LOG-04	Medium	System shall log all reviews submitted, edited, or deleted by users.	NFR-AUDIT-03	User ID and rating stored; content archived if removed
FR-LOG-05	High	System shall log all admin actions including impersonation, account block, and content deletion.	NFR-AUDIT-04	Includes acting admin ID and target ID
FR-LOG-06	Medium	System shall log every endpoint hit (method, IP, status, duration) in access logs.	NFR-PERF-01	Stored in observability platform (e.g., ELK/Grafana)
FR-LOG-07	Medium	System shall log all rate-limit events exceeding threshold per user/IP.	NFR-SEC-05	Used for blacklist logic
FR-LOG-08	Medium	System shall allow querying logs by time range, endpoint, and user ID.	NFR-ADMIN-01	Filter UI in Admin Panel
FR-LOG-09	Medium	System shall store all logs securely with encryption and access control.	NFR-SEC-02	Role-based access to sensitive logs
FR-LOG-10	Medium	System shall store logs for a minimum of 12 months for audit and compliance purposes.	NFR-DATA-RET-01	Configurable log retention policy
FR-LOG-11	Medium	System shall alert dev team if log volume exceeds predefined thresholds (e.g., log spam/flood).	NFR-MON-01	Alert via webhook, Slack, or Opsgenie
FR-LOG-12	Medium	System shall log changes to system configuration or environment variables.	NFR-AUDIT-05	Dev-only changes tracked by deployment tool
FR-LOG-13	Medium	System shall support exporting logs as JSON or CSV for compliance review.	NFR-EXPORT-01	Export triggers in admin dashboard
FR-LOG-14	Low	System shall log all successful and failed 2FA or OTP verification attempts.	NFR-SEC-07	Details include OTP source, expiry, status
FR-LOG-15	Medium	System shall log request/response payloads of system-to-system integrations (scraper, vendor sync).	NFR-INTEGRATION-01	Logs stored in integration log pool
FR-LOG-16	Medium	System shall track file upload/download logs (e.g., KYC documents, terms).	NFR-SEC-08	File name, user ID, and timestamp recorded
FR-LOG-17	Medium	System shall log email/SMS failures and response codes from third-party gateways.	NFR-NOTIFY-04	Includes delivery status, retry attempts
FR-LOG-18	Medium	System shall allow filtering logs by module (auth, offers, products, etc.) for support teams.	NFR-UX-04	Drop-down or tag-based search in logs viewer
FR-LOG-19	Medium	System shall log consent events (e.g., checkbox for marketing or data sharing).	NFR-GDPR-01	Timestamp and user ID associated
FR-LOG-20	Medium	System shall ensure logs are tamper-resistant and time-synced using NTP.	NFR-SEC-09	Includes hash integrity checks

5.13. Notifications

ID	Priority	Description	Related NFR ID	Acceptance Criteria
----	----------	-------------	----------------	---------------------





FR-NOTIF-01	High	System shall notify users by email/SMS when a new offer is received for their request.	NFR-NOTIFY-01	Notification includes offer amount, institution, and CTA
FR-NOTIF-02	High	System shall send a welcome email/SMS upon successful registration.	NFR-NOTIFY-02	Message includes login tips and user support info
FR-NOTIF-03	High	System shall notify institutions when a new user request matches their filter criteria.	NFR-NOTIFY-03	Partner webhook or admin panel alert triggered
FR-NOTIF-04	Medium	System shall send notification when an offer is accepted or declined by the user.	NFR-NOTIFY-04	Status reflected in partner dashboard
FR-NOTIF-05	Medium	System shall alert users about expiring offers 24 hours before expiration.	NFR-TIME-01	Scheduled job triggers message
FR-NOTIF-06	Medium	System shall allow users to manage preferences (opt-in/out of SMS, email).	NFR-UX-02	UI form with toggle for each channel
FR-NOTIF-07	High	System shall record every notification event (type, status, recipient, timestamp) in notification logs.	NFR-AUDIT-02	Logs accessible to admin via dashboard
FR-NOTIF-08	Medium	System shall prevent more than 3 SMS or 5 emails to the same user within 30 minutes.	NFR-RATE-02	Exceeding this triggers suppression logic
FR-NOTIF-09	Medium	System shall retry failed email/SMS delivery up to 3 times with exponential backoff.	NFR-RESILIENCE-01	Status tracked per attempt
FR-NOTIF-10	Medium	System shall support template-based messaging in 3 languages (AZ, RU, EN).	NFR-INTL-01	Template selection based on user language
FR-NOTIF-11	Low	System shall allow exporting logs of notifications for compliance purposes.	NFR-EXPORT-01	Available as CSV with date range filters
FR-NOTIF-12	Medium	System shall allow in-app notifications to be marked as read or unread.	NFR-UX-03	Toggle visible in notification center
FR-NOTIF-13	Medium	System shall show a counter icon for unread in-app notifications.	NFR-UX-03	Count decrements as messages are read
FR-NOTIF-14	Medium	System shall log all delivery responses from external services (Twilio, Mailgun).	NFR-AUDIT-03	Includes HTTP status, delivery ID, and latency
FR-NOTIF-15	Medium	System shall notify admins about system errors (500s, failed jobs) via email/Slack.	NFR-OPS-01	Triggered from monitoring service
FR-NOTIF-16	Medium	System shall allow partners to receive notifications for campaign expirations or usage thresholds.	NFR-PARTNER-01	Visible in partner dashboard
FR-NOTIF-17	Medium	System shall allow configuration of notification channels per event type.	NFR-CONFIG-01	Defined in admin config file or DB
FR-NOTIF-18	Medium	System shall timestamp and store user consent for receiving marketing notifications.	NFR-GDPR-01	Auditable via /admin/consent-log
FR-NOTIF-19	Medium	System shall ensure duplicate messages are not sent to same user within the same batch job run.	NFR-RESILIENCE-02	Deduplication step enforced at job run
FR-NOTIF-20	Medium	System shall support notification previews for admins when testing new templates.	NFR-ADMIN-03	Preview includes substitution of sample data
FR-NOTIF-21	Medium	System shall allow export of opt-out report for regulatory compliance.	NFR-EXPORT-02	List includes user ID, opt-out date, channel

5.14. Admin Panel

ID	Priority	Description	Related NFR ID	Acceptance Criteria
FR-ADMINP-01	High	System shall allow admin users to log into the admin panel with separate auth credentials.	NFR-SEC-01	Token contains admin or super_admin role
FR-ADMINP-02	High	System shall allow viewing, searching, and filtering all users by role, status, or name.	NFR-UX-01	Supports live filters and pagination





FR-ADMINP-03	High	System shall allow admins to block, deactivate, or reset passwords for users.	NFR-SEC-02	Action logged and user notified
FR-ADMINP-04	High	System shall display all product requests with filters for date, type, amount, and user.	NFR-OPS-01	Data grid with sorting options
FR-ADMINP-05	Medium	System shall allow editing or deleting offers flagged for abuse or error.	NFR-MOD-01	Requires reason and confirmation
FR-ADMINP-06	Medium	System shall display all reviews and allow moderation actions (approve, hide, delete).	NFR-REV-02	Moderation log stored with action trace
FR-ADMINP-07	Medium	System shall allow impersonation of users for debugging purposes.	NFR-AUDIT-01	Admin ID and impersonation target logged
FR-ADMINP-08	Medium	System shall allow CSV/JSON export of requests, offers, users, or reviews.	NFR-EXPORT-01	Filtered data included in export
FR-ADMINP-09	Medium	System shall allow configuration of business rules (loan term limits, interest bounds).	NFR-CONFIG-01	UI form allows edits to backend settings
FR-ADMINP-10	Medium	System shall show dashboard widgets (KPIs) such as daily requests, offers, users.	NFR-ANALYTICS-01	Metrics sourced from Analytics module
FR-ADMINP-11	Medium	System shall allow adding/editing/deleting scraped products.	NFR-SCRAPER-01	Edits trigger version tracking
FR-ADMINP-12	Medium	System shall log all changes made through Admin Panel.	NFR-AUDIT-02	Logs are searchable by action or user ID
FR-ADMINP-13	Medium	System shall allow managing institution partner profiles and permission levels.	NFR-INST-01	Role matrix editable from admin UI
FR-ADMINP-14	Medium	System shall allow defining product tags used for filtering or user targeting.	NFR-TAXONOMY-01	Tags stored in metadata system
FR-ADMINP-15	Medium	System shall allow viewing rate-limit and error logs per endpoint/IP.	NFR-SEC-04	Logs pulled from central log service
FR-ADMINP-16	Medium	System shall support multi-language product label editing in the UI.	NFR-INTL-01	Tabs per language, stored to i18n store
FR-ADMINP-17	Medium	System shall allow triggering test notifications for template previews.	NFR-NOTIFY-05	Test sent to admin's inbox
FR-ADMINP-18	Medium	System shall allow access to data retention settings and clean-up tools.	NFR-DATA-RET-01	Configurable purge tools for old logs, expired users
FR-ADMINP-19	Medium	System shall display flagged content summary: suspicious requests, abusive reviews, etc.	NFR-MOD-02	Table with filters and resolution state
FR-ADMINP-20	Medium	System shall allow dynamic filtering of users by request history, approval rate, or last login.	NFR-UX-04	Advanced filters and smart search

5.15. Data Export / Download

ID	Priorit y	Description	Related NFR ID	Acceptance Criteria
FR-EXPORT-01	High	System shall allow admin users to export full lists of user requests in CSV or JSON format.	NFR-EXPORT-01	Filters apply before export
FR-EXPORT-02	High	System shall allow exporting submitted offers (all or filtered) in CSV format.	NFR-EXPORT-02	File includes offer ID, user, institution, interest, timestamp
FR-EXPORT-03	High	System shall allow partner institutions to export offer performance reports.	NFR-PARTNER-01	Includes conversion rates, view count, accepted offers
FR-EXPORT-04	Medium	System shall allow exporting review data by institution/product in CSV.	NFR-REV-01	Export fields: rating, comment, user (masked), date





FR-EXPORT-05	Medium	System shall allow users to export their request history in JSON format.	NFR-UX-02	Accessible from user dashboard
FR-EXPORT-06	Medium	System shall support PDF export of an individual product or offer detail.	NFR-UX-03	PDF includes visual formatting, logo, metadata
FR-EXPORT-07	Medium	System shall allow exporting admin logs by event type (login, block, impersonation).	NFR-AUDIT-01	Available in /admin/logs with filter
FR-EXPORT-08	Medium	System shall support exporting scraped product lists by institution.	NFR-SCRAPER-01	Filters by status, institution, type
FR-EXPORT-09	Medium	System shall support daily scheduled exports of KPI metrics in Excel to admins.	NFR-ANALYTICS-01	Sent via email attachment
FR-EXPORT-10	Medium	System shall export notification logs (status, channel, response code) for compliance review.	NFR-NOTIFY-04	Supports date filtering and batching
FR-EXPORT-11	Low	System shall support export of institution profiles with partner usage statistics.	NFR-INST-01	Includes partner name, activity status, total offers
FR-EXPORT-12	Medium	System shall support GDPR-style user data export including personal profile and request history.	NFR-GDPR-01	Download request fulfilled within 72 hours
FR-EXPORT-13	Low	System shall allow exporting list of flagged reviews/comments for moderation tracking.	NFR-MOD-01	Fields include status, reviewer ID (masked), reason
FR-EXPORT-14	Medium	System shall allow exporting a comparison table of selected products in PDF or Excel.	NFR-UX-04	Columns matched per product type
FR-EXPORT-15	Medium	System shall support downloading anonymized request/offer data for research/AI training.	NFR-AI-01	No PII included; request via admin approval
FR-EXPORT-16	Medium	System shall support export of tax-relevant data for institutional reporting (e.g., interest paid).	NFR-REGULATORY-01	Available quarterly in secured partner zone
FR-EXPORT-17	Low	System shall support Excel-style filtered views during export (e.g., institution = X, type = Y).	NFR-UX-05	Export matches current frontend filters
FR-EXPORT-18	Medium	System shall compress large exports >5MB into ZIP or GZIP format before delivery.	NFR-PERF-01	ZIP auto-download or emailed to user
FR-EXPORT-19	Low	System shall include institution branding on all partner-initiated exports (logo, contact).	NFR-BRANDING-01	Visible in PDF or Excel header
FR-EXPORT-20	Medium	System shall log all export events with user ID, date, export type, and file size.	NFR-AUDIT-02	Searchable in /admin/export-log



6. Non-Functional Requirement

6.1. Security & Compliance

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-PS-001	Must Have	The platform shall support a minimum of 100,000 concurrent users without degradation.	100,000 concurrent sessions	Techleaf SLA, ISO 25010	Load test via JMeter/K6
NFR-PS-002	Must Have	Product search results must be displayed to the user in under 2 seconds in 95% of cases.	$P95 \leq 2s$	UX SLA	Lighthouse, APM dashboards (New Relic)
NFR-PS-003	Must Have	All API responses must return within 800ms under normal load.	Avg API latency $\leq 800ms$	OWASP Performance Guide	API gateway logs, APM tracing
NFR-PS-004	Should Have	The system should auto-scale compute resources to handle up to 5x sudden traffic spikes .	Horizontal auto-scaling enabled	AWS/GCP Autoscaling Policy	K8s HPA logs, AWS/GCP metrics
NFR-PS-005	Must Have	Background jobs (e.g., notifications, scraping) must complete within 2 minutes per batch.	Max duration ≤ 2 mins	Job SLA	Queue monitoring logs
NFR-PS-006	Should Have	Product caching shall be used to reduce DB read load by at least 70% .	$\geq 70\%$ cache hit rate	Redis/Memcached standards	Redis hit/miss dashboard
NFR-PS-007	Must Have	Homepage and landing screens must load in under 1.5 seconds on 4G mobile connection.	$P90 \leq 1.5s$ (4G, mobile-first)	Core Web Vitals	Lighthouse, WebPageTest, Google CWV
NFR-PS-008	Could Have	Frequently accessed APIs (e.g., product listings) should be served via CDN edge caching .	CDN edge hit $\geq 60\%$	CDN Performance Tuning	CDN logs (e.g., Cloudflare, Akamai)
NFR-PS-009	Must Have	Scraping processes must fetch data from each integrated source within 60 seconds .	Avg scrape time $\leq 60s$ per source	ETL SLA	Scraper job logs
NFR-PS-010	Should Have	Real-time notifications (WebSockets/push) must be delivered in under 5 seconds to the client.	$P95$ delivery $\leq 5s$	UX Real-time Metric	Push/WebSocket latency test
NFR-PS-011	Must Have	The platform should maintain $\geq 99.5\%$ response success rate during load tests of 10k RPS.	$\geq 99.5\%$ HTTP 2xx	Techleaf SLA, ISO 25010	Stress test with report
NFR-PS-012	Could Have	Static assets (images, scripts) should be optimized to achieve average load size of $< 1.5 MB$.	Bundle size $< 1.5MB$	Web Vitals, Google CWV	Bundle analyzer, Lighthouse
NFR-PS-013	Should Have	The system should detect and throttle abusive scraping/API usage dynamically.	Throttle abuse over 1000 RPH per IP	OWASP A10	Rate-limit logs
NFR-PS-014	Must Have	Database queries must return within 300ms on average under load.	Avg DB query time $\leq 300ms$	ISO 25010	APM DB metrics, slow query logs



NFR-PS-015	Must Have	Offer submission process (form + backend process) must be completed in ≤ 3 seconds .	Full offer TX ≤ 3s	UX Flow SLA	Frontend UX log, APM
NFR-PS-016	Could Have	AI modules (e.g., recommendation engine) should complete			

6.2. Performance & Scalability

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-PS-001	Must Have	System must support ≥100,000 concurrent users without latency > 1s for any core function.	P99 latency ≤ 1s @ 100k users	SLA, ISO 25010	Load testing (Locust, JMeter)
NFR-PS-002	Must Have	API average response time must remain ≤800ms for all core endpoints.	Avg latency ≤ 800ms	OWASP Perf Guide	APM tools (Datadog, NewRelic)
NFR-PS-003	Must Have	Search results must appear in <2s for 95% of user queries.	P95 ≤ 2s	UX SLA	Synthetic UX test
NFR-PS-004	Must Have	Pages must load fully in ≤3s on 4G mobile.	Page Load ≤ 3s on avg.	Google Core Web Vitals	PageSpeed, Lighthouse, GTmetrix
NFR-PS-005	Should Have	System should autoscale compute pods based on CPU >70% or memory >75%.	Scaling latency ≤ 60s	AWS / GCP Standards	K8s HPA & cluster metrics
NFR-PS-006	Must Have	AI recommendation response must be delivered in ≤1s (P95).	Inference time ≤ 1s	ML Serving SLA	APM logs, AI gateway metrics
NFR-PS-007	Must Have	Platform must tolerate up to 5x load spike without full restart.	Auto-healing response ≤ 2 mins	High Availability	Resilience tests
NFR-PS-008	Should Have	Homepage LCP (Largest Contentful Paint) should be ≤2.5s for 90% of visitors.	LCP ≤ 2.5s	Google CWV	Lighthouse, WebPageTest
NFR-PS-009	Should Have	Caching must reduce product list DB calls by ≥70% .	≥70% Redis hit rate	Cache Strategy Docs	Redis/Memcached analytics
NFR-PS-010	Must Have	Critical services (Auth, Offers, Requests) must have uptime of ≥99.9% monthly .	≥99.9% uptime	SLA Agreement	Uptime robot, Grafana
NFR-PS-011	Could Have	Image and static asset sizes should not exceed 1.5MB per page .	Bundle size < 1.5MB	Web Performance Best Practices	Asset analyzer, compression test
NFR-PS-012	Must Have	System should process an offer submission end-to-end in ≤5s (form+API+response).	End-to-end ≤ 5s	UX SLA	User scenario test
NFR-PS-013	Must Have	DB query execution time should average ≤300ms under 80% CPU load.	DB avg latency ≤ 300ms	DB Tuning SLA	PostgreSQL/SQL Profiler
NFR-PS-014	Should Have	Daily scraped job (batch updates) should complete in <15 minutes across 30+ institutions.	≤15 mins for all batch jobs	ETL/Sync Benchmark	Scheduler logs



NFR-PS-015	Must Have	System must maintain 99.95% success rate across 1 million requests per day.	≤0.05% error rate	SLA	HTTP 5xx logs, error tracing
NFR-PS-016	Should Have	Partner dashboards should load analytics summaries ≤4s for datasets <500k rows.	Summary render ≤ 4s	BI SLA	Query logs, UI load time
NFR-PS-017	Should Have	WebSocket updates (e.g., offers) must be delivered in ≤5 seconds from event trigger.	Push latency ≤ 5s	Real-Time Comm Best Practices	WebSocket logs, trigger tests
NFR-PS-018	Could Have	Enable multi-region failover to replicate traffic in ≤180 seconds in case of zone outage.	DR Recovery Time ≤ 3 mins	ISO 22301, Business Continuity	Chaos testing

6.3. User Experience & Accessibility

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-UX-001	Must Have	Techleaf must be fully responsive and render correctly on screens from 320px to 1920px.	100% mobile & desktop responsive	WCAG 2.1, Google RWD	BrowserStack, device emulation
NFR-UX-002	Must Have	All pages must load in ≤3 seconds on 4G and ≤2s on broadband.	Page Load ≤ 3s mobile, ≤2s desktop	UX SLA, Web Vitals	GTmetrix, Lighthouse
NFR-UX-003	Must Have	Platform must achieve WCAG 2.1 AA compliance for accessible content rendering.	100% tag and role compliance	WCAG 2.1 AA	WAVE, axe-core, manual screen reader test
NFR-UX-004	Should Have	Real-time typeahead suggestions should appear within 500ms of input.	Autocomplete latency ≤ 0.5s	UX Standard	Input simulation, timer logs
NFR-UX-005	Must Have	Support 3 language options (AZ, RU, EN) across full UI, including error messages.	100% translation coverage	i18n QA	Manual QA per locale
NFR-UX-006	Should Have	Support dark mode toggle from user profile settings.	Persistent theme toggle	UX Theme Guidelines	Functional test, localStorage session
NFR-UX-007	Must Have	All form inputs must support field validation and show inline messages within 300ms .	Inline feedback ≤ 300ms after user input	UX Forms Standard	Field input tests, usability QA
NFR-UX-008	Should Have	Platform should support keyboard navigation for all key features .	Tab+Enter covers all major flows	WCAG 2.1	Tab test / manual UX flow
NFR-UX-009	Could Have	Voice search should be enabled for finding financial products on mobile app.	Result ≤ 2s from speech input	Voice UX Experiments	Voice-to-text integration test
NFR-UX-010	Should Have	Review writing UX should support real-time character count and live sentiment feedback.	UX response ≤ 300ms	Review UX Framework	Test review form, timer logs
NFR-UX-011	Must Have	Platform should provide emoji rendering support in reviews/comments without affecting layout.	Emojis rendered on all browsers	Social UX Guideline	Browser UX validation
NFR-UX-012	Could Have	Users should be able to “favorite” or bookmark products across sessions.	Bookmark persistence = local + server state	UX Personalization	Session + DB state test



NFR-UX-013	Must Have	Form inputs must not block accessibility readers or cause form errors with screen readers.	Form tags must have ARIA labels	WCAG 2.1, ADA	Screen reader simulation
NFR-UX-014	Should Have	All interactive elements must have tap area $\geq 48 \times 48$px on mobile.	100% pass for touch targets	Google Mobile Usability Guide	Mobile emulator test

6.4. System Reliability & Backup

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-SR-001	Must Have	Platform availability must be $\geq 99.9\%$ monthly , excluding scheduled maintenance.	≤ 43.2 minutes downtime/month	SLA, ISO 22301	Uptime monitor (StatusCake, Pingdom)
NFR-SR-002	Must Have	Full database backup must occur automatically every 6 hours .	Incremental: 1h, Full: 6h	ISO 27001 A.12.3	Cron logs, retention audit
NFR-SR-003	Must Have	Backup files must be encrypted with AES-256 and stored in a different availability zone.	Dual-region encrypted backups	CIS Benchmark, ISO 27002	Backup storage inspection, checksum validation
NFR-SR-004	Must Have	Recovery Time Objective (RTO) in case of major failure must be ≤ 10 minutes .	$RTO \leq 10$ mins	ISO 22301	Disaster recovery drill
NFR-SR-005	Must Have	Recovery Point Objective (RPO) must be ≤ 1 hour .	Data loss ≤ 60 minutes	ISO 22301	Recovery simulation
NFR-SR-006	Should Have	All APIs must have retry logic for transient failures, with 3 retries max + exponential backoff.	Retry delay up to 30s total	API Resilience Guideline	Retry mechanism unit test
NFR-SR-007	Should Have	A real-time failover mechanism should automatically redirect traffic on node crash.	Failover latency ≤ 60 seconds	HA Architecture	K8s readiness probes, NGINX failover test
NFR-SR-008	Could Have	Anomaly detection system should notify about potential downtime 15 minutes in advance .	Forecast error alert: 15 min before	AI Ops / Predictive Recovery	Monitoring + ML test
NFR-SR-009	Should Have	All microservices must support graceful shutdown within 5 seconds during redeploy.	Shutdown latency ≤ 5 s	K8s Pod Lifecycle Management	Shutdown simulation logs
NFR-SR-010	Must Have	Backup retention policy must preserve: daily (14 days), weekly (6 weeks), monthly (12 months) .	Rolling backup archive	GDPR, ISO 27001	Backup file inspection, policy audit
NFR-SR-011	Should Have	Load balancer must perform health checks every 10 seconds , removing unhealthy nodes in ≤ 30 s.	Detection delay ≤ 30 s	AWS/GCP Load Balancing	Load balancer config inspection
NFR-SR-012	Must Have	Scraper job failures must be logged and retried automatically within 2 minutes .	Retry window ≤ 120 s	ETL Reliability SLA	Job log audit
NFR-SR-013	Should Have	Error budget must not exceed 0.1% downtime monthly (complement to uptime target).	Downtime ≤ 43.2 mins/month	SRE Principle	SLA tracker
NFR-SR-014	Must Have	Redis cache and message queues must support high availability failover (sentinel/replication).	Redis Sentinel or equivalent enabled	Redis HA Strategy	Sentinel heartbeat check



NFR-SR-015	Could Have	Admin dashboard should display live system health from 3 key services (API, DB, Queues).	3 live indicators with latency stats	Internal UX Guide	Healthcheck UI component test
NFR-SR-016	Must Have	Logs must be streamed to centralized logging system with 99.99% delivery rate .	Log ingestion $\geq$ 99.99%	ISO 27001 Logging	ELK/Grafana integration test

6.5. Compliance & Legal

ID	Priority	Description	Target / Metric	Standard / Regulation	Validation Method
NFR-CL-001	Must Have	Platform must not store credit card data and shall only integrate with PCI-DSS certified providers .	Zero credit card data in DB/API logs	PCI DSS 3.2.1, GDPR	PCI compliance report, code audit
NFR-CL-002	Must Have	Techleaf must be registered with local regulatory authority (e.g., ARFM) to offer financial matching services.	Proof of license & registration visible	ARFM Azerbaijan	Compliance audit log
NFR-CL-003	Must Have	All personal data handling must comply with GDPR, ISO/IEC 27701, and Azerbaijani privacy laws .	100% data lifecycle compliance	GDPR, ISO 27701, National Law	Privacy audit / DPA review
NFR-CL-004	Must Have	Users must explicitly consent to data processing during onboarding.	100% checkbox opt-in stored with timestamp	GDPR Art. 6, Consent Registry	Consent database inspection
NFR-CL-005	Should Have	Platform should allow users to request data export of their personal data (GDPR Art. 20).	JSON/CSV download within 72 hours	GDPR Data Portability	Manual export test, user portal UX
NFR-CL-006	Must Have	Legal disclaimers must be shown before finalizing any product comparison or offer acceptance.	Shown in 100% of financial product flows	Financial Transparency Laws	UI walkthrough + test automation
NFR-CL-007	Should Have	All product offers must be verified by partner institutions via Admin Panel before becoming visible.	Manual verification step enforced	Financial Fair Advertising Rules	Admin audit logs, offer status API check
NFR-CL-008	Must Have	User's right to be forgotten (GDPR Art. 17) must be implemented.	Data deletion upon request $\leq$ 30 days	GDPR Right to Erasure	Data erasure workflow test
NFR-CL-009	Should Have	All offer documents and reviews must retain immutable timestamps for audit/legal defense.	ISO 8601 + cryptographic hash	Digital Integrity Laws	Record hashing + timestamp match
NFR-CL-010	Must Have	All agreements, disclosures, and terms must be available in AZ, RU, EN languages.	100% translation of legal pages	Multilingual UX Law	Translation QA / Language toggle validation
NFR-CL-011	Must Have	All data processors used by Techleaf must have a signed DPA (Data Processing Agreement) .	100% 3rd parties under DPA	GDPR Art. 28, ISO 27001 BCP	Vendor list + contract register
NFR-CL-012	Must Have	An updated and accessible Privacy Policy & Terms of Use must be available on the website at all times.	Linked on all public pages	GDPR Art. 13, ePrivacy Directive	Footer check, 404 link test
NFR-CL-013	Should Have	An internal whistleblower mechanism should exist for legal violations or data mishandling.	Anonymous channel + report tracking	ISO 37001, Ethics Best Practice	Whistleblower tool audit



NFR-CL-014	Must Have	Platform must notify regulators within 72 hours of confirmed personal data breaches.	Breach notification $\leq 72h$	GDPR Art. 33, ISO 22301	Incident simulation + policy review
NFR-CL-015	Could Have	Platform should offer users a history of accepted disclosures and consents.	Consent journal available on profile	GDPR, UX Legal Compliance	Profile page test
NFR-CL-016	Must Have	Cookie consent and tracking controls must follow ePrivacy Directive and GDPR Cookie Law .	Granular controls + opt-out	GDPR Recital 30, ePrivacy 2002/58/EC	Cookie banner test / browser storage audit
NFR-CL-017	Should Have	Admins must be able to export a compliance audit log for any user within 48 hours.	Log export $\leq 48h$	GDPR / Regulatory Monitoring	Log export UX test, CSV/PDF export
NFR-CL-018	Could Have	Partner institutions should receive automated reminders if their offer T&Cs are not updated every 6 months.	T&C age check and alert	Internal Legal Hygiene Policy	Cronjob notification logic

6.6. Availability & Monitoring

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-AM-001	Must Have	Platform must be available 24/7 with $\geq 99.9\%$ uptime per month.	Downtime ≤ 43.2 min/month	ISO 22301, SLA	Uptime monitor (Pingdom, StatusCake)
NFR-AM-002	Must Have	Critical system components must emit liveness and readiness probes for orchestration.	All microservices with K8s probes	Kubernetes Best Practices	Kubernetes probe inspection
NFR-AM-003	Must Have	A centralized logging system must collect logs from 100% of backend services .	Log ingestion rate $\geq 99.99\%$	ISO 27001 Logging	ELK / Loki / Fluentd ingestion test
NFR-AM-004	Must Have	Monitoring must include metrics on CPU, memory, disk, API errors, DB latency, and queue depth.	Dashboard with 100% component observability	SRE Observability Standard	Grafana / Datadog dashboard validation
NFR-AM-005	Must Have	Alerting must trigger within 30 seconds of any critical service failure.	P90 alert latency $\leq 30s$	ISO 22301, Incident SLA	Alert simulator / PagerDuty test
NFR-AM-006	Should Have	Alerts must be delivered via at least 2 independent channels (e.g., SMS + Slack).	Dual-channel alert delivery	Multi-channel Alerting Policy	Redundancy test
NFR-AM-007	Must Have	Any unhandled 5xx error rate above 1% for 5 minutes must trigger a major incident.	$\geq 1\%$ 5xx over 5 mins $\rightarrow$ Alert	Internal SLO Breach Rule	Alert rule configuration audit
NFR-AM-008	Should Have	Monitoring dashboards must be accessible by authorized personnel with role-based access .	RBAC-enabled dashboard access	SOC 2 Type II	Access control review
NFR-AM-009	Must Have	System must store minimum 90 days of logs and 30 days of metrics for incident forensics.	90d logs, 30d metrics	GDPR, ISO 27001	Retention policy inspection
NFR-AM-010	Should Have	All alerts must be tagged with priority: Critical, Warning, Informational .	100% alert classification	ITIL Incident Management	Alert rule metadata check



NFR-AM-011	Could Have	Platform should implement synthetic monitoring for end-to-end transaction flow validation.	Daily synthetic test of key flows	QA / Reliability QA	Selenium + K6 e2e checks
NFR-AM-012	Must Have	All availability metrics must be visualized on a real-time dashboard (Grafana, Datadog).	Dashboard refresh $\leq 15s$	SRE Visualization Standard	Manual check + latency test
NFR-AM-013	Should Have	Health of 3rd party APIs (banks, insurers) must be monitored and visualized.	Partner API healthcheck every 1 min	SLA Monitoring	Ping + fallback response checker
NFR-AM-014	Could Have	Weekly availability reports should be automatically sent to key stakeholders.	Report delivery every Monday 9 AM	Internal DevOps SOP	Scheduled report delivery test
NFR-AM-015	Should Have	Incident history and downtime logs should be retained for 1 year .	12 months archival	ISO 22301, Legal Records	Archive inspection
NFR-AM-016	Must Have	Load balancer and gateway services must support active-passive failover within 30 seconds .	Failover switch $\leq 30s$	HA Infra Design	Load balancer failover test
NFR-AM-017	Must Have	Internal metrics such as uptime, error rate, and latency must be available per service .	Per-service APM dashboards	Microservice Monitoring	Grafana panel check

6.7. Maintainability & DevOps

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-MD-001	Must Have	The platform must support zero-downtime deployments via rolling updates.	No downtime during deploy	Kubernetes, DevOps Best Practice	K8s deployment test, API continuity checks
NFR-MD-002	Must Have	All services must follow 12-factor app principles for scalability and maintainability.	100% factor adherence	12factor.net	Code audit checklist
NFR-MD-003	Must Have	Techleaf must support automated CI/CD pipelines with build, test, and deploy stages.	CI/CD coverage $\geq 95\%$ of repos	GitOps / DevOps Practices	CI/CD dashboard audit
NFR-MD-004	Must Have	Platform must allow rollback to previous version within 5 minutes of deployment failure.	Rollback latency ≤ 5 mins	Kubernetes / Helm Rollback	Helm rollback simulation
NFR-MD-005	Should Have	Each microservice should be independently deployable and versioned.	100% service isolation	Microservice Architecture Principles	Deployment test per module
NFR-MD-006	Must Have	All infrastructure changes must be defined as Infrastructure as Code (IaC) .	100% IAC coverage for cloud resources	Terraform / Pulumi	Code repository validation
NFR-MD-007	Must Have	Pre-deployment integration tests must run for every major feature or release.	$\geq 95\%$ PRs must pass integration stage	CI Quality Gates	CI dashboard, test report
NFR-MD-008	Should Have	A canary deployment strategy should be used for critical updates.	$\leq 10\%$ traffic during initial rollout	Progressive Delivery Practice	Canary routing config review



NFR-MD-009	Must Have	Dev, Test, and Prod environments must be fully isolated and configuration-managed.	3 isolated environments	ISO 27001, Environment Segregation	Network check, config drift detection
NFR-MD-010	Must Have	Configuration values (e.g., secrets, API keys) must be externalized and stored securely .	100% configs in Vault / Secrets Manager	OWASP Config Management	Code review + vault access logs
NFR-MD-011	Must Have	Platform must support blue-green deployments for major releases.	Downtime $\leq 0s$ during version switch	Release Management Best Practices	Deployment routing simulation
NFR-MD-012	Should Have	System must automatically tag builds with semantic versioning (e.g., 1.0.3).	All builds semantically versioned	DevOps Standards	Build log inspection
NFR-MD-013	Could Have	Developers should receive deployment preview links for QA before production merges.	Preview link for each feature branch	Preview-Driven Dev	Preview URL test
NFR-MD-014	Must Have	All production services must log structured logs in JSON format with correlation IDs.	100% structured logs	Logging Standards	Log review + correlation trace test
NFR-MD-015	Must Have	System must be able to scale from 3 to 30 pods per service within 2 minutes .	Scaling time ≤ 2 mins	Horizontal Scalability SLA	K8s autoscale simulation
NFR-MD-016	Should Have	DevOps must maintain runbooks and playbooks for each critical service.	Docs for top 20 services	SRE / ISO 22301	Documentation check
NFR-MD-017	Must Have	CI/CD pipelines must enforce code linting, formatting, and security scanning .	100% enforced checks	DevSecOps Guidelines	Pipeline config + scanner output
NFR-MD-018	Should Have	CI/CD failures must notify responsible developers within 1 minute of failure.	Notification latency $\leq 60s$	Alerting Best Practices	Slack/email test delivery
NFR-MD-019	Could Have	Automated documentation generation should occur post-merge to main branch.	Docs built within 1 min of merge	Code Quality Standard	CI documentation check
NFR-MD-020	Must Have	Daily build health metrics must be available to engineering managers.	Health report (pass %, avg time, failures)	Engineering Metrics Practice	Jenkins/GitHub Actions report audit

6.8. Auditability & Traceability

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-AT-001	Must Have	All user authentication attempts (success/failure) must be logged with timestamp and IP.	100% of auth events logged	ISO 27001 A.12, OWASP Logging	Auth log review
NFR-AT-002	Must Have	Every action taken by admin or partner (e.g., impersonation, review approval) must be audit-logged .	100% sensitive actions recorded	SOC 2 Type II	Audit log API check
NFR-AT-003	Must Have	All logs must contain traceable correlation IDs for session and request tracking.	100% of request logs contain correlation_id	Distributed Tracing Standard	Trace testing via APM (e.g., NewRelic, Jaeger)



ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-AT-004	Should Have	Every user-generated review must retain immutable metadata: IP, time, browser fingerprint.	Metadata embedded and unalterable	Legal Recordkeeping Standards	DB record inspection
NFR-AT-005	Must Have	All system logs must be write-once, immutable, and tamper-proof .	WORM (write-once-read-many) retention enabled	ISO 27001 Logging Integrity	Elastic/Kibana index immutability check
NFR-AT-006	Should Have	Each loan/insurance request must be assigned a globally unique identifier (UUID) .	100% traceable via request_id	ISO 8601 UUID traceability	UUID format + response test
NFR-AT-007	Must Have	All user-submitted documents (e.g., ID photos) must store an audit trail of upload and access history .	File log with uploader, viewer, timestamp	GDPR, Financial KYC Traceability	Access audit logs, file metadata check
NFR-AT-008	Must Have	Offer lifecycle must include a full log of every status change (created → approved → rejected, etc.).	Minimum 5-stage traceability per offer	Financial Service Logging Standard	Workflow log diff
NFR-AT-009	Must Have	System must log and alert on deletion of any record by admin or backend script.	Real-time delete event + alert	Admin Forensic Policy	Deletion test, alerting validation
NFR-AT-010	Should Have	Logs must be searchable by user ID, email, or request ID for debugging and compliance.	≤ 2 seconds search latency in log viewer	DevOps Observability Practice	Search functionality test
NFR-AT-011	Must Have	All user consents and terms acceptance actions must be timestamped and stored for ≥ 5 years .	Retention policy ≥ 5 years	GDPR Consent Archiving	DB retention check, export test
NFR-AT-012	Could Have	Logs should support export to external audit partners as CSV/JSON for quarterly reviews.	Export format available for 3rd parties	External Audit Compliance	Export feature test in Admin Panel
NFR-AT-013	Must Have	Changes in configuration settings (e.g., interest rate cap, filters) must be logged with editor identity.	Config change log with diff	ISO 27001 A.9	Admin config UI event logging
NFR-AT-014	Should Have	Partner institutions should have access to their own audit logs within the partner dashboard.	Role-restricted log visibility	Role-Based Access Control	RBAC test + audit UI review
NFR-AT-015	Must Have	All scheduled jobs (cron, scrapers) must record execution status, output summary, and failure logs.	100% job audit traceable	DevOps Job Management SOP	Job runner log inspection
NFR-AT-016	Should Have	Each exported file must contain a footer with export time, admin ID, and checksum.	Hash footer + metadata tag	ISO 27001 / Data Integrity	Export file QA
NFR-AT-017	Could Have	All logs should be retained in cold storage (e.g., S3 Glacier) for up to 7 years .	Archival policy = 7 years	Regulatory Recordkeeping Policy	Cloud storage policy review
NFR-AT-018	Must Have	Alerts must be generated for failed log ingestion or missing events in critical flows.	Log delivery success ≥ 99.99%	Log Reliability Compliance	Missing entry detection + Prometheus alerting

6.9. Data Quality & Integrity





ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-DQ-001	Must Have	All data received from web scrapers or partners must pass a data validation schema check .	100% entries pass schema (e.g., price: number)	ISO 8000, OWASP Data Validation	JSON schema test, type assertion logs
NFR-DQ-002	Must Have	Product offers from partners must have unique identifiers to prevent duplicates.	≥ 99.99% ID uniqueness	Data Consistency Best Practices	UUID collision check, duplicate logs
NFR-DQ-003	Must Have	Techleaf must detect and eliminate duplicate records during import or sync jobs.	Duplicate ratio ≤ 0.1%	ETL Data Cleanliness Rules	Data diff & deduplication routine
NFR-DQ-004	Must Have	All critical numeric fields (e.g., interest rate, duration) must be range validated .	100% of fields have min/max check	OWASP Input Validation	Validation rules + negative test scenarios
NFR-DQ-005	Should Have	Displayed financial data must match the source (bank/insurer site) within 30 seconds of update .	Data freshness gap ≤ 30s	Financial API Freshness	Diff monitor + time delta comparison
NFR-DQ-006	Must Have	User-submitted information (e.g., salary, age) must be type and format validated before saving.	≥ 99.5% valid input fields	UX Field Validation Policies	Form validator + backend assert test
NFR-DQ-007	Should Have	All dropdowns and selection fields must reference controlled vocabularies or enums .	100% dropdowns use backend enum sync	API-Driven UI Consistency	Enum mismatch detection
NFR-DQ-008	Must Have	Data syncing failures (scraping, API pull) must generate error logs + retry mechanism .	100% failures logged, retry within 5 minutes	ETL Recovery Logic	Retry queue monitor
NFR-DQ-009	Must Have	Deleted or hidden offers must not be returned in product search or user dashboards.	Ghosted entry visibility = 0%	Logical Data Lifecycle Enforcement	DB query and UI visibility test
NFR-DQ-010	Should Have	Product data should support versioning for rollback and historical auditing.	Versioned schema enabled	Immutable Data Logging	Version tag + rollback test
NFR-DQ-011	Must Have	All data pipelines (scraping, user, partners) must maintain idempotency to avoid over-writes.	Safe re-run = no unintended changes	DataOps Best Practices	Job replay test
NFR-DQ-012	Must Have	Sensitive inputs (e.g., income, address) must be stored exactly as submitted , without transformation.	100% field-level fidelity	Data Fidelity Standard	Field diff comparison, PII check
NFR-DQ-013	Could Have	Automated scoring system should assign a data integrity score to each record based on trust levels.	Score range: 0–100	AI/ML Quality Enhancement	Data scoring module + histogram
NFR-DQ-014	Must Have	Data received from banks must be marked as official / scraped in system tags.	Tag compliance = 100%	Internal Source Attribution Policy	Tag inspection per record
NFR-DQ-015	Should Have	All dropdown / UI field values must match the backend reference table with <1% mismatch tolerance .	UI-backend mismatch rate < 1%	UX Data Sync Compliance	Daily diff report
NFR-DQ-016	Could Have	Data validators should provide context-aware error messages for invalid input fields.	All errors show specific reasons	UX Clarity Metric	Form submission A/B test





NFR-DQ-017	Must Have	All product offers must be logically consistent (e.g., min amount < max amount, rate > 0%).	Logical validation pass rate $\geq$ 99.99%	Product Logic Assurance	Constraint ruleset test
NFR-DQ-018	Should Have	Partner-uploaded data (CSV/XML) must pass format validation before processing.	Upload errors $\leq$ 0.5%	File Validation Guide	Upload test with malformed input
NFR-DQ-019	Must Have	Empty/null fields in mandatory sections must trigger hard stop validation errors .	Null rate = 0% on required fields	Field Completeness Standard	Null injection testing
NFR-DQ-020	Should Have	AI product match predictions must log confidence score per result.	Prediction output: 1-100% confidence	Explainability in AI Systems	API response inspection

6.10. Internationalization & Localization

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-IL-001	Must Have	Techleaf must support Azerbaijani (AZ), Russian (RU), and English (EN) languages.	100% translation coverage in 3 languages	I18n Standard, UX Localization	UI toggle test, translation QA
NFR-IL-002	Must Have	Language preferences must be persisted per user account and session .	Language cookie or DB preference stored	UX Personalization Policy	Cookie check, locale test
NFR-IL-003	Must Have	All system messages, error alerts, and validations must be translatable.	100% locale key coverage for message catalog	ISO 9241-171	Message bundle audit, fallback test
NFR-IL-004	Must Have	Financial content (currency, date, % rate) must follow user's regional formatting conventions .	Locale-specific format applied correctly	Unicode CLDR / Intl standards	Currency + date formatting tests
NFR-IL-005	Should Have	User-submitted content (e.g., reviews) must display in user's interface language , if translations exist.	Language-aware fallback logic enabled	Social UX Best Practice	Language context display test
NFR-IL-006	Must Have	Default platform language for anonymous users must match browser language or geolocation .	Auto-detect + allow manual override	Geo-IP and User-Agent Localization	IP toggle test, language switch A/B test
NFR-IL-007	Should Have	All legal pages (Terms, Privacy Policy) must be fully localized in AZ, RU, and EN.	100% translated legal docs	GDPR Art. 13, UX Legal Compliance	Content review for all locales
NFR-IL-008	Must Have	Number input fields must accept locale-aware separators (e.g., comma vs. dot) and normalize internally.	Locale normalization logic enabled	Form Input Localization Standard	Input testing per locale
NFR-IL-009	Should Have	Admin Panel must support multilingual labels and tooltips for cross-border admins.	AZ + EN minimum required	Admin UX Policy	Manual review per locale
NFR-IL-010	Could Have	Content from institutions (product descriptions) should be auto-translatable with fallback support .	Translation enabled or flag "missing"	NLP Translation Framework	Auto-translate API test





NFR-IL-011	Must Have	Mobile and web apps must use same language context across all screens and forms.	No mismatch in mobile/web language state	Cross-Platform UX	QA across all screens
NFR-IL-012	Should Have	User profile forms must adapt label and input direction to language direction (e.g., LTR/RTL) if needed.	RTL support if ever extended	I18n UX Accessibility	UI orientation test
NFR-IL-013	Must Have	All platform-wide constants (country names, currencies) must use backend-synced translation keys .	100% enum sync with translation repo	API-Driven I18n	Sync validation script
NFR-IL-014	Should Have	Product filtering/search labels must be shown in selected language without layout breakage .	Fully localized filters UI with dynamic resizing	Responsive I18n Standards	Mobile/tablet layout test
NFR-IL-015	Could Have	Partner portal and offer input forms should eventually support RU/EN in addition to AZ.	At least RU/EN UI fields supported	B2B Localization Roadmap	Roadmap QA

6.11. Testability & QA Automation

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-TQ-001	Must Have	All backend services must maintain a unit test coverage of $\geq 80\%$.	$\geq 80\%$ lines/functions tested	ISO/IEC 29119	Coverage report (e.g., Istanbul, Jest)
NFR-TQ-002	Must Have	All frontend components must be testable using component-level tests (e.g., React Testing Library) .	$\geq 70\%$ UI unit coverage	Frontend QA Guidelines	Vite/Jest/Cypress report
NFR-TQ-003	Must Have	All public and internal APIs must have automated contract tests using OpenAPI/Swagger.	100% Swagger endpoints covered	API Governance Policy	Dredd / Postman / Swagger test logs
NFR-TQ-004	Must Have	QA must maintain end-to-end test suites covering at least 90% of core flows.	90% business-critical scenarios	E2E Test Strategy (Selenium, Playwright)	CI pipeline pass rate
NFR-TQ-005	Should Have	Test runs must complete in <15 minutes to prevent pipeline bottlenecks.	End-to-end test latency ≤ 15 mins	CI Efficiency Metric	CI timer / execution report
NFR-TQ-006	Must Have	Each new feature PR must pass unit, integration, and security scans before merge.	100% gated merges	Secure SDLC Pipeline	CI gate policy
NFR-TQ-007	Must Have	Mock servers (e.g., for partner bank APIs) must be available for test environments.	Mocked 100% of 3rd-party services	API Simulation Strategy	Mock endpoint audit
NFR-TQ-008	Should Have	The QA team must be able to run parallel tests across environments (Dev, Test, Staging).	Multi-env parallelism enabled	Test Orchestration Policy	Execution scaling check
NFR-TQ-009	Must Have	A test data seeding strategy must be in place for each environment.	Resettable, anonymized, GDPR-compliant sets	Test Data Management Standard	Seeding log check



NFR-TQ-010	Should Have	System must auto-generate test reports with screenshots & logs on failure.	100% test run visibility	QA Best Practice	CI report logs / visual regression checks
NFR-TQ-011	Must Have	All business-critical processes (e.g., loan request, product compare, onboarding) must have E2E flows .	100% critical flows tested	Techleaf Core QA Requirements	QA matrix coverage report
NFR-TQ-012	Should Have	Mobile version must be tested across iOS and Android using real devices/emulators .	Devices: iPhone SE → iPhone 14 Pro, Pixel 6+	Mobile QA Policy	BrowserStack / Firebase lab logs
NFR-TQ-013	Must Have	All regression test cases must be tracked in a test case management system (e.g., TestRail).	100% critical regression scenarios documented	ISO 25010	Test case tracker export
NFR-TQ-014	Could Have	The system should support A/B testing with measurable control/test group isolation.	Experimentation framework live in Prod	UX Experimentation Policy	Toggle + metrics tracking
NFR-TQ-015	Should Have	All UI forms must be tested for input field edge cases and browser compatibility .	Fields tested across 3 major browsers	UX QA Guidelines	Browser matrix execution
NFR-TQ-016	Should Have	Scrapers and ETL jobs must have validation tests to detect malformed or missing fields.	100% batch validation rule coverage	ETL QA Best Practice	Scraper pipeline test suite
NFR-TQ-017	Could Have	AI-based recommendations must be validated with expected output range + confidence interval .	Confidence logging in test mode	ML QA Protocol	Model A/B prediction monitor
NFR-TQ-018	Must Have	Every CI/CD stage must stop the release if critical test failures or security vulnerabilities occur.	Block merge on fail	DevSecOps Release Policy	Pipeline config review
NFR-TQ-019	Must Have	QA automation must run at least 4x per day (on every branch or merge).	4+ automated builds per day	CI/CD Run Frequency Goal	GitHub Actions / Jenkins dashboard
NFR-TQ-020	Should Have	Uptime of test environments (Dev, QA, Staging) must be ≥98% to ensure test schedule continuity.	Downtime ≤ 14.4 hours/month	Internal QA Availability Policy	Environment uptime monitor

6.12. Interoperability & Integration

ID	Priority	Description	Target / Metric	Standard / Compliance	Validation Method
NFR-II-001	Must Have	All Techleaf APIs must support RESTful architecture with standard HTTP methods and status codes.	100% RESTful API compliance	OpenAPI 3.0, REST Best Practices	Swagger schema audit
NFR-II-002	Must Have	APIs must be documented in OpenAPI/Swagger format and accessible in both dev and test environments.	100% documented endpoints	Developer Experience (DX)	Swagger UI verification
NFR-II-003	Must Have	All integrations (e.g., bank scraping, insurer APIs) must log	100% integration events logged	ISO 27001 Logging	Log audit trail





		source, timestamp, and status for traceability.			
NFR-II-004	Must Have	System must support partner-specific field mappings and data adapters for incoming offers.	Mapping modules for all active partners	Integration Adapter Framework	Mapping test per partner
NFR-II-005	Should Have	Techleaf should implement a partner sandbox API environment for new partner testing.	Sandbox available for all certified partners	Fintech Onboarding Policy	Sandbox API exposure test
NFR-II-006	Must Have	System must consume external APIs using token-based or OAuth 2.0 authentication .	100% secure token exchange	OWASP API Security Top 10	Token expiry + refresh test
NFR-II-007	Must Have	Scrapers must convert scraped data to Techleaf's internal schema before storage or display.	100% ETL schema conformance	Internal Data Normalization	Schema mapping + output validation
NFR-II-008	Must Have	All webhook consumers must implement event signature verification to prevent spoofed calls.	Signature match $\geq$ 100%	Webhook Security Policy	HMAC / token header validation check
NFR-II-009	Should Have	Platform must expose selected public APIs for third-party fintech tools and aggregators .	Authenticated public API with rate limits	Open Banking / B2B Ecosystem	Rate limiting & auth test
NFR-II-010	Should Have	Data exchange with regulatory bodies (e.g., eGov, tax, sanction lists) must use standard XML/JSON schema .	Format: JSON or ISO 20022 XML	Government API Integration	Schema validation test
NFR-II-011	Must Have	Any 3rd-party partner integration must be deployable in a configurable, modular way (no code redeploy) .	Plug-and-play architecture	Microservice Isolation	Feature flag & env-based config check
NFR-II-012	Should Have	Partner offer ingestion must include acknowledgment messages for success/failure of records.	ACK for 100% API calls to partners	Partner SLA Integration Guideline	Offer tracking logs
NFR-II-013	Could Have	Platform may support real-time streaming (e.g., Kafka, Webhooks) for live partner updates.	Low-latency offer stream setup	Streaming Data Architecture	Throughput test + offset replay check
NFR-II-014	Should Have	System must support versioned APIs to avoid breaking integrations during updates.	v1, v2 routes concurrently supported	API Lifecycle Management	Version routing test via Postman
NFR-II-015	Must Have	Partner-specific business logic must reside in isolated configuration modules , not core logic.	100% partner logic decoupled	B2B Customization Strategy	Logic review in config folders/modules
NFR-II-016	Must Have	All outbound integrations must include timeouts, retries, and circuit breakers .	Timeout: $\leq$ 10s, Retries: 3, CB threshold: 5 fails	Resilient Integration Pattern	Circuit breaker test with simulated failure
NFR-II-017	Must Have	Response format must be consistent across all internal APIs (e.g., data, error, pagination).	JSON: { success, data, error, pagination }	OpenAPI Best Practice	Response schema validation
NFR-II-018	Should Have	Partners should be able to query their own usage/activity logs via a Partner API Portal .	Activity logs available via dashboard/API	B2B Transparency UX	Partner login session test





7. System Requirements

7.1. Platform & Device Compatibility

ID	Requirement Description	Constraint / Target	Applies To	Notes / Justification
SR-001	System shall support access via Chrome, Firefox, Safari, and Edge browsers.	Latest + 1 previous major version	Web Frontend	Ensures cross-browser functionality for widest user adoption
SR-002	System shall provide responsive layout across all screen sizes.	Width range: 320px to 2560px	Web Frontend	Supports both mobile-first users and desktop institutions
SR-003	System shall be compatible with iOS 13+ and Android 9+ mobile operating systems.	Fully functional UI on iPhone SE – 14+, Pixel 5 – 8+	Web App on Mobile	Ensures platform works on most widely used smartphones
SR-004	System shall support functionality on Windows 10/11, macOS (Monterey+), and Ubuntu 20.04+.	Full Admin Panel functionality	Admin / Ops Team	Required for in-house teams and enterprise users
SR-005	System shall not require any native installation and shall run fully in web browsers.	100% web-based delivery	End-users / Admin	Reduces installation barriers, updates are centralized
SR-006	All clickable or interactive UI elements shall be minimum 48x48px on mobile devices.	Mobile tap target size $\geq$ 48px	Web / Mobile UI	Improves mobile usability and complies with Google UX guidelines
SR-007	UI shall support multi-touch gestures (scroll, tap, zoom) on mobile/tablet devices.	No gesture conflict	Mobile Web View	Ensures smooth user interaction across devices
SR-008	System shall support landscape and portrait orientation without layout breakage.	Layout integrity across orientation switch	Web & Mobile Frontend	Required for mobile and tablet landscape users
SR-009	Language selection (AZ, RU, EN) must be consistently visible on all resolutions.	Header or menu must expose toggle	Frontend UI	Supports multilingual access from all devices
SR-010	System shall auto-detect browser language and serve default locale accordingly for new sessions.	Fallback: English	Session Handler	Improves onboarding and localized UX experience
SR-011	System shall render at ≥ 30 FPS on mid-range mobile devices for scroll and form interactions.	Sustained FPS $\geq$ 30	Web Mobile UI	Smooth animation and input without jank
SR-012	System shall maintain visual consistency across Retina, HD, and 4K displays.	UI scaling = consistent fonts/icons/images	Web Frontend	Important for branding and professional appearance
SR-013	System shall support touch screen and keyboard inputs simultaneously (hybrid devices).	Multi-input behavior allowed	Windows Tablets, Laptops	Enables productivity on devices like Surface, iPad with keyboard
SR-014	All input fields and forms shall be optimized for mobile keyboards (e.g., numeric type for salary).	Input type = number, email, tel, etc.	Mobile UI Forms	Streamlines input for faster completion
SR-015	External content such as embedded iframes or third-party widgets shall remain responsive.	Embed max width = 100%, height auto-adjust	Frontend / Widget UI	Ensures consistency of integrated live chats, surveys, etc.



SR-016	System shall not require Flash or other deprecated technologies in any browser.	HTML5 and JavaScript only	Entire Application	Prevents obsolescence and improves security
SR-017	Platform shall render print-optimized pages for downloads (e.g., PDF exports).	A4 layout support, no visual overflow	Reports / Documents	Needed for financial product exports and legal confirmations
SR-018	System shall allow users to upload/download files from mobile browsers and desktop browsers .	File picker must support touch interface	Frontend File Modules	Important for KYC, review documents, and offer comparisons
SR-019	Cookies, session storage, and local storage shall be fully supported and cleared when user logs out.	Session isolation per device/browser	Auth & Storage Layer	Prevents leak of session data across shared or public devices
SR-020	Accessibility support shall comply with WCAG 2.1 AA across all supported browsers.	≥ 95% accessibility compliance	Frontend & QA	Required for regulatory and ethical digital inclusivity
SR-021	All downloadable content (PDF, CSV, XLSX) must be openable in Microsoft Office 2016+ and Google Workspace .	Cross-compatibility ensured	Admin Panel / Exports	Prevents format compatibility issues for institutional use
SR-022	Frontend shall support multiple tab usage with consistent session behavior.	Shared session token, CSRF isolation	Frontend / Auth Layer	Needed for users comparing multiple products in different tabs

7.2. System Architecture & Deployment

ID	Requirement Description	Constraint / Target	Applies To	Notes / Justification
SR-023	System shall be designed using a microservices architecture with service isolation.	Each core module is independently deployable	Backend / API Gateway	Enables modular scaling and fault containment
SR-024	System shall be deployed using Docker containers and orchestrated with Kubernetes.	Docker 24.0+, K8s 1.27+	DevOps / All Environments	Standard for scalable and reproducible deployments
SR-025	The infrastructure shall support three environments : Dev, Test, and Production.	Isolated clusters with separate databases	DevOps	Prevents data bleed and allows safe experimentation
SR-026	All microservices shall expose APIs over HTTPS only , with TLS 1.2+ enabled.	TLS 1.2 or above enforced	API Gateway / Reverse Proxy	Ensures secure communication across services
SR-027	APIs shall be stateless , with session information stored in Redis or JWTs.	No in-memory session persistence	Backend / Auth Services	Required for horizontal scaling
SR-028	All deployments must be executed via CI/CD pipelines with linting and testing stages.	100% Git-based deployment	DevOps / Engineering	Enforces secure and controlled deployments
SR-029	Services shall follow semantic versioning (semver) for every release.	Format: X.Y.Z (e.g., 1.4.2)	Backend / APIs	Enables clear update tracking and rollback
SR-030	Every service shall be health-check enabled with /health and /readiness endpoints.	Standardized endpoint for K8s probes	Backend Services	Required for orchestration and monitoring
SR-031	Each deployment shall support rolling updates with no downtime.	Zero-downtime via K8s rolling strategy	DevOps / CI/CD	Ensures platform stays available during releases
SR-032	All configurations shall be managed via environment variables or secure config maps.	100% 12-factor compliance	Backend / DevOps	Keeps secrets and logic separated





SR-033	The system shall use NGINX or equivalent reverse proxy for routing and load balancing.	Load balancing at ingress level	Infrastructure Layer	Ensures scalability and resilience
SR-035	The system shall support auto-scaling based on CPU $\geq 70\%$ or memory $\geq 75\%$.	HPA configuration per pod	DevOps / K8s	Required for elastic scaling under load
SR-036	A dedicated API gateway shall manage routing, throttling, and request validation.	Rate-limiting, CORS, auth middleware	Backend / External APIs	Centralizes control and external-facing access
SR-037	A service registry and discovery mechanism shall be used (e.g., DNS or Consul).	DNS-based or registry-based service resolution	Microservice Communication	Prevents hardcoded IPs and improves resilience
SR-038	System must support graceful shutdown to avoid data loss during pod restarts.	Shutdown hooks implemented per service	All Backend Services	Required for stable deployments and rollbacks
SR-039	Every build artifact shall be uniquely tagged and stored in a container registry .	Git SHA or release version-based tags	DevOps / Build Infra	Enables repeatability and traceability
SR-040	Logs shall be forwarded from containers to a centralized ELK or Loki + Grafana stack .	Real-time log streaming	DevOps / Logging	Enables observability and root cause analysis
SR-041	All background jobs (e.g., scraping, batch scoring) must run in job-specific worker pods .	Isolated execution from main service pods	Scheduler / K8s	Improves fault isolation and performance tuning
SR-042	The application shall use Redis for caching and short-lived session token storage.	Redis cluster with failover	Auth / AI Cache	Enhances speed and handles volatile storage needs
SR-043	All external services shall be integrated via well-documented interface contracts .	OpenAPI-based internal specs	Backend / Integration Teams	Reduces integration risk and improves dev onboarding
SR-044	System shall support feature toggles to control beta/experimental rollouts.	Toggle flags via DB or ConfigMap	Frontend / Backend Feature Dev	Enables safe A/B testing and rollback
SR-045	System must support blue-green deployments for high-stakes releases.	Prod-v1 and Prod-v2 routing supported	DevOps / Critical Features	Minimizes production risk during major launches

7.3. Data Handling & Integration

ID	Requirement Description	Constraint / Target	Applies To	Notes / Justification
SR-046	System shall synchronize scraped financial product data from partner websites.	Sync interval: every 30 minutes	Scraper / Data Engine	Ensures fresh offers and accurate display
SR-047	Scraped data shall be normalized into Techleaf's internal schema before ingestion.	Schema versioning via ETL pipelines	Backend / Data Layer	Maintains uniform structure across sources
SR-048	Partner API integrations must comply with JSON or XML (ISO 2002 preferred) .	Preferred: JSON, fallback: XML	Integration / Backend	Ensures broad compatibility with institutions
SR-049	All date and time fields shall be stored in UTC and displayed using user-localized format.	UTC in DB, local in UI	Backend / Frontend	Prevents timezone errors and aligns with global practices





SR-050	Currency values must support international formatting and round to 2 decimal places.	Localized decimal separators, e.g., 1.234,56 → 1,234.56	UI / Reports / Backend	Necessary for multi-locale usability
SR-051	All incoming offer data must be validated using schema-based validators (e.g., JSON Schema).	100% of imported offers pass schema checks	Backend / Scraper ETL	Avoids incomplete or malformed records
SR-052	Product and offer records must include source tagging (scraped, api, manual).	Source stored in metadata	Backend / Audit Logs	Required for traceability and auditability
SR-053	API responses must include metadata fields: timestamp, source, and version.	Part of response envelope	External API Consumers	Enables partner sync logging and integrity validation
SR-054	All partner API credentials must be stored in encrypted key vaults , not in environment files.	Use AWS Secrets Manager or Vault	DevOps / Integration	Prevents accidental leaks and ensures secret rotation
SR-055	Techleaf must not store user credit card or CVV information locally.	Use tokenized PCI-compliant gateway	Compliance / Payments	Required for PCI DSS scope reduction
SR-056	The scraping engine must detect changes in HTML DOM structure and log them.	Structural diff engine reports changes	Scraper Engine / Alerts	Prevents silent scraping failures due to website redesigns
SR-057	Offer deduplication logic must run before data is indexed for search or displayed.	UUID or checksum-based deduping	Backend / Indexer	Avoids duplicate offers from multi-source pulls
SR-058	AI scoring and recommendation models must log input source, model version, and output .	Versioned AI inference log	AI Service / Backend	Enables reproducibility and audit of decisions
SR-059	Offer data imported from files (CSV/XML) must pass format validation before entry.	Accept/reject with error messages	Admin Panel / Partner Import	Enhances quality and reduces manual cleaning
SR-060	Each request submitted by a user must receive a globally unique identifier (UUIDv4) .	Format: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx	Backend / Notification Logs	Enables traceability and partner tracking
SR-061	The system shall retry failed API calls to partner systems up to 3 times with backoff.	Retry logic with exponential delay	Backend Integration Layer	Improves resilience against network fluctuations
SR-062	File upload fields must accept: .jpg, .png, .pdf, .docx, with max 10MB limit .	MIME type check and server-side validation	Frontend / File Processor	Ensures security and consistency in document submissions
SR-063	Web scraping components shall support proxy rotation and rate-limiting control .	Limit: ≤ 5 requests/sec per domain	Scraper Infrastructure	Avoids IP bans and enables responsible data retrieval
SR-064	All scraped HTML content shall be parsed using language-aware NLP (AZ/RU/EN) extractors.	Language detection + field-based extractor	AI / NLP Pre-Processor	Ensures accuracy in multilingual content parsing
SR-065	Manual data entries by admins must go through the same backend validators as scraped/API data.	Shared validation middleware	Admin Panel / Validation	Prevents bypassing business logic through manual entries

7.4. Security & Access Control

ID	Requirement Description	Constraint / Target	Applies To	Notes / Justification
SR-066	All user passwords must be hashed with bcrypt (minimum 12 salt rounds) .	No plaintext or MD5/SHA-1	Auth System / Database	Prevents password theft even in case of DB breach





SR-067	All authentication tokens must be signed JWTs , expiring within 60 minutes .	JWT expiry $\leq$ 1 hour	Backend / API Auth	Ensures session safety and allows token rotation
SR-068	Refresh tokens must be stored using HttpOnly, Secure cookies , separate from access tokens.	2-token model separation	Frontend / Auth Layer	Prevents token theft via XSS
SR-069	Platform shall implement Two-Factor Authentication (2FA) using TOTP (e.g., Google Authenticator).	Optional but available for all users	Web & Admin Interface	Protects user accounts and admin logins
SR-070	Admin and internal roles must follow role-based access control (RBAC) with privilege boundaries.	Role types: Viewer, Editor, Approver, Auditor	Admin Panel / Backend	Enforces least privilege model
SR-071	All sensitive user actions (loan submit, data update) must trigger audit-logged events .	Event recorded with user ID + timestamp	Backend / Logs / Admin Panel	Enables compliance and forensic traceability
SR-072	Every API endpoint shall validate authentication and authorization per route.	100% coverage in middleware	Backend / API Gateway	Prevents unauthorized access to internal logic
SR-073	Login attempts must be rate-limited to 5 failed attempts per 10 minutes .	Lockout policy or CAPTCHA after limit	Auth Service / UI	Protects against brute-force login attacks
SR-074	CSRF tokens must be implemented for all state-changing frontend requests.	CSRF token per session	Frontend $\rightarrow$ Backend	Prevents cross-site request forgery
SR-075	System shall log and alert on suspicious activity patterns , e.g., login from new country.	Geo-IP alerting + email confirmation	Backend / Security Layer	Detects account takeover or fraud attempts
SR-076	Secrets, tokens, and credentials shall be stored in secure secrets manager or vault .	No env files or hardcoded tokens	DevOps / Deployment Layer	Ensures compliance with secure DevOps
SR-077	Admins must be able to revoke user or partner access instantly , across all environments.	Token blacklist or session destroy	Admin Panel / User Mgmt	Allows emergency access revocation
SR-078	Session cookies must include flags: Secure, HttpOnly, and SameSite=Lax or Strict.	Set-Cookie policy enforced	Web UI / Auth Backend	Prevents CSRF, XSS, and session leakage
SR-079	Session timeout must be enforced after 15 minutes of inactivity , with warning at 12 min.	Idle timeout = 15 min	Web UI	Enhances account protection
SR-080	API keys issued to external clients must be scoped, rate-limited, and revocable .	Key scope: read, write, admin	API Gateway / Partner API	Allows secure external consumption of services
SR-081	All admin actions (add/remove offer, edit T&C) must be confirmed via a second factor or audit PIN .	Admin critical action control	Admin Panel	Prevents internal misuse or errors
SR-082	Login and password reset workflows must use email-based OTPs with 6-digit expiration in 5 minutes.	OTP = 6 digits, TTL = 5 mins	Auth Service	Ensures secure account recovery
SR-083	Security headers must be included in all HTTP responses (X-Content-Type, X-Frame-Options).	Hardened headers checklist	API Gateway / Frontend	Prevents browser-based injection and clickjacking
SR-084	The system must not allow mixed content (HTTPS pages loading HTTP resources).	HTTPS enforced for all assets	Frontend / CDNs	Prevents downgrade and man-in-the-middle attacks





SR-085	All infrastructure access (e.g., SSH) must be key-based only , no password logins allowed.	SSH public key access only	DevOps / Cloud Infra	Hardens server access controls
SR-086	All user-uploaded files must pass a virus/malware scan before storage or preview.	AV engine: ClamAV or AWS Inspector	File Upload / Storage	Prevents malicious file injections
SR-087	Backend must enforce input validation & output encoding for all dynamic fields.	OWASP Secure Input guidelines	All backend APIs	Prevents injection, XSS, and broken parsing
SR-088	Admin roles and permissions shall be reviewed quarterly and upon staff exit.	Review report archived	Compliance / IT Admin	Reduces insider threat exposure
SR-089	User accounts must be automatically locked after 30 days of inactivity (optional config).	Lock period: 30 days	Auth / Account Services	Prevents ghost accounts from being hijacked

7.5. Compliance & Observability

ID	Requirement Description	Constraint / Target	Applies To	Notes / Justification
SR-090	The system must maintain 99.9% monthly uptime excluding planned maintenance.	≤ 43.2 minutes downtime/month	DevOps / Backend	Required for financial-grade SLA guarantees
SR-091	System logs must be stored for a minimum of 90 days , with secure archival up to 1 year.	Hot storage: 90d, cold archive: 365d	ELK / Loki / S3 Archive	Needed for legal and operational auditability
SR-092	The system shall support automated uptime monitoring with alerts for outages or errors.	Alert latency ≤ 30 seconds	Monitoring / DevOps	Ensures incident response capability
SR-093	Every major release must be documented with changelog, rollback plan, and version ID .	Document per semver release	Engineering / DevOps	Facilitates audit trails and system transparency
SR-094	System shall comply with WCAG 2.1 AA accessibility standards across web and mobile.	≥ 95% WCAG test pass rate	Frontend	Required for inclusive access and legal compliance
SR-095	Platform shall expose a /status endpoint for external uptime and health checks.	Returns system health metrics	API Gateway / Monitoring	Enables partner observability and 3rd-party status integration
SR-096	All user consents, disclaimers, and terms acceptances must be stored with immutable logs .	Consent timestamp, user ID, IP	Backend / Legal Logs	Required by GDPR and financial transparency rules
SR-097	All audit logs must be write-once, tamper-proof , and stored with hash-based integrity.	SHA-256 or equivalent proof	Audit Trail / Logging Infra	Prevents manipulation of critical audit records
SR-098	System shall publish a privacy policy, cookie policy, and terms of service in 3 languages.	AZ, RU, EN versions available	Legal / UX	Mandatory for multi-locale financial platforms
SR-099	Platform must generate a monthly operations report with stats (users, offers, uptime).	Delivery to admins via email or panel	Admin Panel / Ops Reporting	Enables executive oversight and transparency
SR-100	Logs must be enriched with request ID, user ID, service name, and timestamp .	Standardized format for log search	Logging Service / DevOps	Enables easy correlation across services
SR-101	All external API integrations must maintain a call log with status, latency, and errors .	Call-level diagnostics retained	Partner API / Backend Logs	Required for SLA tracking and debugging



SR-102	Platform shall comply with GDPR, ISO 27001, and local data protection law in Azerbaijan.	No PII stored without consent	Legal / Security Team	Required for operations in EU and local regulatory environment
SR-103	Daily system backup status must be logged, timestamped, and verified via email alerts .	Verified daily at 01:00 system time	Backup Infrastructure	Ensures backup readiness and legal defensibility
SR-104	Each admin and QA role must have activity logs stored for minimum 6 months .	Immutable and exportable	Admin Panel / QA Tools	Required for accountability in controlled environments
SR-105	Techleaf must conduct a quarterly log review for anomalies and policy violations .	Reviewed by Security + Ops	CISO / DevOps / Compliance Team	Prevents long-term unnoticed breaches or misuse
SR-106	The system shall expose internal metrics via Prometheus for pod-level and route-level observability.	CPU, memory, latency, errors	Monitoring Stack / Grafana	Supports capacity planning and fault analysis
SR-107	All error codes must be documented and categorized in the API reference.	Codes: 2xx, 4xx, 5xx with meaning	Backend / API Docs	Enables developers and partners to debug effectively

8. Development Requirements

8.1. Technology Stack

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DR-001	Backend microservices shall be written in Python 3.11+ using FastAPI for APIs.	All MS (Auth, Product, Search...)	Async architecture + REST-native	Aligns with scalable microservice pattern
DR-002	Frontend shall be built using React 18+ , TypeScript , and Tailwind CSS .	Web and Admin Panels	Strict typing, atomic UI structure	Enables modular UI and error-safe code
DR-003	Admin panel shall use Next.js with ShadCN UI and full SSR for performance.	Admin Interface	Next.js 13+, server components	Optimized internal user experience
DR-004	Microservices must communicate over REST APIs , with optional WebSocket for events.	All MS, Notification	REST for commands, WS for events	Ensures decoupling and reactive design
DR-005	PostgreSQL 15+ shall be used for relational data across all core modules.	Product, Auth, User, Orders	PostGIS if location needed	Chosen for ACID, JSONB, reliability
DR-006	MongoDB Atlas (M10+) shall be used for document-based storage (scraped HTML, AI data).	Web Scraper, Analytics	Index for URL + updatedAt	For flexible ETL and fast reads
DR-007	Redis 7+ will store sessions, OTPs, and temporary offer recommendations .	Auth, Notification, Search	TTL tokens ≤ 15 mins	Caching + secure token storage
DR-008	Queue and background jobs shall be handled using Celery + RabbitMQ .	Scraping, Notifications	Retry, rate-limit support	Manages async job queues
DR-009	Web scraping shall use Playwright (Python) with proxy rotation and failover .	Web Scraper	Headless Chromium, stealth	Required for scraping real banks
DR-010	AI-based scoring models shall use scikit-learn, LightGBM , and be deployed as APIs.	Analytics & AI MS	Input schema = standard loanRequest	Matches users to optimal products
DR-011	All microservices shall be containerized using Docker 24+ and deployed on Kubernetes 1.27+ .	All Backend Services	Helm charts versioned by env	Ensures consistency and scale



DR-012	Monitoring and alerting shall be implemented using Prometheus + Grafana .	DevOps, All MS	Custom metrics: latency, retries, failures	Enables proactive incident detection
DR-013	API documentation must follow OpenAPI 3.1 and be published via Swagger UI or Redoc .	All Exposed APIs	Autogen from FastAPI routes	Ensures external developer access
DR-014	Message schema must follow JSON with snake_case for keys.	All APIs and Events	No camelCase or inconsistent casing	Promotes uniform data contracts
DR-015	Identity & login integration with 2FA and JWT must be standard across all services.	Auth MS, Admin Panel	TOTP with device remember feature	Required for financial-grade security

8.2. Code Quality & Standards

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DR-016	All code must conform to PEP8 (Python) or Airbnb ESLint + Prettier (JavaScript) .	Python / JS Repos	Lint errors = CI fail	Prevents syntax inconsistencies
DR-017	TypeScript must be enforced for all frontend code, no any types allowed.	React Frontend	Full type coverage	Improves reliability and refactoring
DR-018	Directory structures must follow feature-based modular structure .	All frontend/backend	One folder per feature	Aids readability and testability
DR-019	Every function and endpoint must include docstrings or JSDoc comments .	Backend / Frontend	Coverage $\geq 90\%$	Enables onboarding and tool integration
DR-020	All APIs must include status codes , input/output schema, and error messages.	All Public APIs	OpenAPI coverage = 100%	Guarantees predictable API usage
DR-021	Git commits must follow Conventional Commits (feat; fix; refactor: etc.)	All Code Contributors	Used for changelog gen	Enables semantic versioning and CI tagging
DR-022	Code must have $\geq 80\%$ unit test coverage for backend and frontend logic.	All Modules	CI gates fail below 80%	Ensures regression resistance
DR-023	Each PR must pass CI build, lint, test, and scan gates before merge.	All Branches	PR template enforced	Enforces pre-deployment stability
DR-024	Shared libraries (e.g., validators, auth handlers) must be documented and reused.	Internal Repos	No duplication allowed	Encourages modular, DRY codebase
DR-025	Environment configs (e.g., .env, secrets.yaml) must never be committed.	All Repos	.gitignore and CI audit	Enforces secrets hygiene

8.3. Development Environment

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DR-026	Developers shall use Docker Desktop for local development environments.	Local Development	Docker v24+, Docker Compose v2+	Ensures consistency between local and production environments
DR-027	The system must be configured to allow local debugging via VSCode + PyCharm .	Backend / Frontend	VSCode 1.70+, PyCharm 2022+	Provides full-feature debugging capabilities
DR-028	The system shall use GitHub/GitLab for version control, with feature-branching .	All Development	Git, GitHub Actions, GitLab CI	Centralized version control and streamlined collaboration





DR-029	All environment variables should be stored in .env files, with sample files (.env.example).	Backend / Frontend	Avoid secrets in public repositories	Simplifies local environment setup and prevents sensitive info leaks
DR-030	Development environments shall include mock APIs or stubs for all external integrations (banks, payment providers).	Backend / API Testing	Mock API for external services	Ensures isolated dev environment and avoids dependencies on external services
DR-031	Developers should use Prettier (for JS/TS) and Black (for Python) for automatic code formatting.	Backend / Frontend	Prettier 2.6+, Black 21.12+	Enforces consistent code styling across teams
DR-032	Local databases must be provisioned via Docker containers with sample data for dev purposes.	DB Dev Setup	PostgreSQL 15+, MongoDB 5.x	Ensures consistent dev testing setup and easy database resets
DR-033	All services must be configured to run with local mock payment gateways for testing purposes.	Payment MS	Fake payment API	Enables complete testing without involving real financial transactions
DR-034	The development environment shall have an auto-reload feature for frontend (React) and backend (FastAPI).	Frontend / Backend	Auto-refresh, Hot Module Replacement (HMR)	Increases developer productivity by instantly reflecting code changes
DR-035	The development environment must support feature flagging for experimenting with new features locally.	Feature Dev / Experimentation	Feature flag toggle via config	Improves flexibility during feature development without code changes

8.4. CI/CD & Release Strategy

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DR-036	All code must pass linting, unit tests, and security scans in the CI pipeline.	All Repositories	Lint: ESLint/Flake8, Tests: pytest, Security: Snyk	Prevents code quality regressions and vulnerabilities during CI
DR-037	The system shall use GitFlow or GitHub Flow for managing feature development and releases.	Codebase / Version Control	Features: dev → feature branches → PR merge into main	Ensures stable versioning and orderly branching strategy
DR-038	Every merge request (PR) must pass unit tests, integration tests, and UI tests before it can be merged.	Backend / Frontend	Coverage: ≥ 80%	Ensures high quality and no broken builds
DR-039	The system shall use Docker images for consistent deployment across dev, test, and prod.	All Environments	Docker v24+	Ensures consistency between dev/test/prod environments
DR-040	Releases must follow semantic versioning (semver) with tags in Git.	Backend / APIs	Format: v1.2.0, v1.2.1	Enables easy rollback and clear release tracking
DR-041	Automated tests must run on every push to the dev branch and pass before merging to main.	Backend / Frontend	CI server: GitHub Actions, GitLab CI, Jenkins	Prevents regressions, builds on latest code
DR-042	All services shall be deployed using Helm charts in Kubernetes for reproducible deployments.	DevOps / Microservices	Helm 3+, chart version pinned	Makes deployments predictable and trackable
DR-043	Staging environment shall mirror production, and must be available for feature testing before release.	Testing / Staging	Staging should be 100% clone of prod	Ensures that all bugs are caught before the final deployment



DR-044	The system must support blue-green or canary releases for zero-downtime production upgrades.	Production	Zero-downtime during critical deployments	Reduces the risk of downtime during critical updates
DR-045	Rollbacks to the previous stable version must be possible with a single command in the CI pipeline.	Production	CI/CD: Manual rollback process via kubectl or Helm CLI	Minimizes downtime in case of critical production issues
DR-046	All deployable artifacts (Docker images) must be versioned and tagged automatically in CI.	DevOps / Build Pipelines	Version tags should include date, commit SHA	Provides traceability of all releases and builds
DR-047	All commits must trigger a version bump for major, minor, or patch changes based on commit messages.	CI Server / Versioning	CI script: version bump when PR merged to main	Enforces disciplined versioning via commit-based rules
DR-048	A rolling deployment strategy must be adopted for production releases, ensuring minimal traffic disruption.	Production	Pod-by-pod upgrade	Ensures controlled rollout and real-time user availability

8.5. Collaboration & Branching

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DR-049	System must allow users to export financial product data in CSV, PDF, and Excel formats.	Product Comparison, User Data	CSV, XLSX, PDF export formats	Common formats for data portability and use in business reports
DR-050	User data export files must contain all user interaction and transaction data within the last 12 months.	User Management, Payments	Maximum file size: 50MB	Compliance for financial auditing and reporting
DR-051	All exportable files should support compression (ZIP) to facilitate large data transfers.	Backend / Reporting	File max size: 500MB per ZIP file	Improves download times and handling of large data sets
DR-052	All financial offer data exports must include a timestamp for the offer's last update.	Admin Panel, Product MS	Date format: YYYY-MM-DD HH:mm:ss	Ensures data freshness and auditability
DR-053	System shall allow users to select date ranges for data exports (e.g., quarterly, yearly).	Product Comparison, Reports	Range: Last 7 days, Last 30 days, Custom date range	Customizable export options for user flexibility
DR-054	Export files must be accessible via secure download links , with expiration after 24 hours .	User Data, Reports	Expiration: 24 hours after generation	Prevents unauthorized long-term access to sensitive data
DR-055	All exported files must exclude sensitive personal data such as passwords, and encrypted values.	User Data, Payment Details	Personal info redacted	Compliance with GDPR and other data protection regulations
DR-056	All API-based exports must support real-time data extraction and allow paging for large datasets.	Reporting, Analytics	Pagination limits: 1000 rows per API call	Prevents server overload when exporting large datasets
DR-057	Export templates for financial data (CSV, PDF) should be pre-formatted to match user expectations.	Finance Reports, Admin	Predefined field order and grouping	Standardizes data presentation for external use (e.g., tax filing)
DR-058	System must allow the downloading of export logs to track request metadata and errors.	Admin Panel, Data Exports	Log retention: 30 days	Necessary for tracking failed exports and providing support
DR-059	Data integrity checks (e.g., SHA256 hashes) must be applied to ensure accurate file downloads.	Backend / Data Export	File checksum verification on download	Ensures that data has not been altered during export or transmission



DR-060	System shall allow manual export of any financial product data directly via Admin Panel.	Admin Panel	Admin role access only	Enables internal data access control and export functionality
--------	---	-------------	------------------------	---

8.6. Tooling & IDE Support

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DR-061	The system must follow GitFlow for feature development, release management, and hotfixes.	Entire Techleaf Development	Feature branches for new functionality	Ensures a consistent and structured branching process
DR-062	Developers must create feature branches for each new task and merge via pull requests (PRs) .	Backend / Frontend	PR approval required before merging to main or dev	Promotes modular and testable code changes
DR-063	PR reviews must be completed by at least one senior developer and one QA team member .	Development, QA	All PRs must pass review	Ensures code quality and reduces defects before merging
DR-064	All critical hotfixes must be merged directly into the main branch and deployed immediately.	Critical Bug Fixes	Only for production-critical issues	Reduces downtime for urgent issues
DR-065	Merge conflicts must be resolved before pushing to any branch; all conflicts must be logged .	Development Workflow	Conflicts must be resolved locally	Prevents unstable code from being pushed
DR-066	System must enforce a "no direct commits to main" policy and use PR-based workflows.	All Development	PRs must always go through dev → staging → prod	Prevents direct code pushes and maintains stable production releases
DR-067	CI/CD pipelines shall run on every push to any feature or dev branch before merging.	All Repositories	Pipelines must pass before merging	Automates quality checks and ensures no broken builds
DR-068	A changelog must be maintained via Conventional Commits for all new features and changes.	Backend / Frontend	Format: feat, fix, docs	Creates automated release notes and clear versioning
DR-069	System shall enforce a code freeze for 2 days prior to production release .	Production Deployments	Freeze enforced 2 days before release	Prevents last-minute changes and bugs right before release
DR-070	All merge conflicts must be logged and resolved by the developer who created them.	Backend / Frontend	Merge log + conflict resolution documented	Promotes responsibility and avoids ignored conflicts

9. Database Requirements

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-001	Techleaf must use PostgreSQL 15+ as the primary relational database for structured data.	Core Data	PostgreSQL 15+	Industry-standard relational database, supports ACID compliance
DB-002	MongoDB shall be used for unstructured data (e.g., logs, AI models).	Data Processing / Logs	MongoDB v5.x+	NoSQL flexibility for unstructured data
DB-003	The database schema must support multi-tenancy to allow future expansion of services.	Backend Core Data	Multi-tenant schema, isolation per tenant	Enables scalability and isolation for different financial products





DB-004	Sharding must be used for distributing large data sets across multiple servers to manage traffic and load.	Backend / Data Layer	Sharded by region or data volume	Ensures scalability and reduces load on individual servers
DB-005	Materialized views must be used for reporting queries to enhance performance and reduce query load.	Reporting / Analytics	Materialized views on high-volume reports	Improves query performance and reduces load on the primary database
DB-006	The database must support real-time data streaming to handle live transaction data efficiently.	Transaction Data	Real-time streaming tools (e.g., Kafka)	Critical for real-time processing and decision-making
DB-033	The system shall use schema-based partitioning for large transactional data (e.g., orders).	Backend / Orders	Partitioning based on date or user region	Optimizes query performance and data management
DB-034	The system shall have separate databases for testing, development, and production to ensure isolation.	Testing / Development	Isolation via Docker containers or separate cloud instances	Prevents accidental data loss or modification in production

9.1. Data Security & Encryption

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-007	All user data must be encrypted using AES-256 encryption when stored in the database.	User Data / Transactions	AES-256 encryption standard	Protects sensitive data and complies with security regulations
DB-008	Payment data (e.g., credit card) should never be stored in the database and should be tokenized.	Payment Data	Tokenization of sensitive payment information	PCI-DSS compliance and enhanced security for financial data
DB-009	SSL/TLS must be used to encrypt data in transit between services and the database.	All Data	Encrypted using TLS v1.2+	Ensures data privacy during transmission
DB-035	Database backup encryption should use AES-256 to ensure data privacy in offsite storage.	Backup Infrastructure	AES-256 encryption for backup files	Ensures security of data during the backup process
DB-036	System must ensure database keys are rotated every 90 days for enhanced security.	Encryption Keys	Key rotation interval: every 90 days	Reduces risk from long-term key exposure
DB-012	Database logs must include encryption key usage information to track key lifecycle events.	Backup / Logging	Log key usage with timestamp	Enables key lifecycle auditing and regulatory compliance

9.2. Data Integrity & Validation

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
----	-------------------------	------------	---------------------	-------------------





DB-013	Foreign key constraints must be enforced across all related tables to ensure relational integrity.	Backend Core Data	Foreign keys: e.g., user_id , product_id	Maintains consistency and prevents orphaned data
DB-014	The system shall implement data normalization (3NF) for all transactional data.	Backend Core Data	3NF compliance required	Avoids data redundancy and ensures efficient query performance
DB-015	All data imports (e.g., financial products, user data) must be validated for correctness and completeness before being stored.	Data Import / Scraping	Validation: format, range, completeness	Ensures accurate and clean data in the database
DB-016	Unique constraints must be enforced for critical fields (e.g., email, transaction ID).	User Data / Transactions	Unique constraint on fields	Prevents duplication of critical information (e.g., email, transaction ID)
DB-017	Data deduplication must be performed automatically for any data entries with matching identifiers.	Data Import / User Records	Deduplication rules based on unique keys (e.g., user_id)	Reduces storage bloat and ensures accurate records
DB-018	The system must implement field-level validation for user input (e.g., email format, transaction amount).	User Input / Transactions	Use of validation libraries (e.g., Joi, Zod)	Ensures valid data entry before storing in database
DB-037	The system shall implement automatic data integrity checks for all imported data at the point of ingestion.	Data Import / Scraping	Auto-validation upon receipt of data	Ensures data integrity and catches issues early in the ingestion pipeline
DB-038	Transactional consistency checks must be applied for financial records before any transactions are committed to the database.	Financial Data	ACID properties: Atomicity, Consistency, Isolation, Durability	Ensures data consistency in high-volume financial transactions

9.3. Backup

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-019	The system must support automated backups every 6 hours , with a retention period of 30 days.	Database / Backup Infrastructure	Automated backup every 6 hours, retention: 30 days	Ensures recovery of critical data in case of system failure
DB-020	Backups must be stored in a geographically separated location for redundancy and disaster recovery.	Cloud Storage / Backup	Offsite storage (e.g., AWS S3, Google Cloud Storage)	Ensures disaster recovery in case of regional failures
DB-021	Backup files must be compressed and stored in secure, encrypted formats (e.g., ZIP, AES).	Backup Infrastructure	Backup file size limit: 100 GB max	Reduces storage space usage and ensures encrypted backups
DB-022	The system must support point-in-time recovery (PITR) for restoring data from any backup.	Database / Production	PITR must work within 10 minutes of last transaction	Enables fast recovery from any point in time



DB-023	Database snapshots must be taken before and after any major deployment or migration.	Production Environment	Snapshots to be taken before and after each deployment	Prevents data loss during major system changes
DB-039	Real-time replication must occur between primary and secondary databases to ensure minimal data loss.	Production DB	Real-time replication, synchronous	Minimizes data loss during disaster recovery
DB-040	System shall use full backups and incremental backups to optimize backup storage and recovery times.	Backup Infrastructure	Full backup weekly, incremental daily	Optimizes backup storage and ensures efficient data recovery

9.4. Replication & High Availability

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-024	The database must be configured for master-slave replication to ensure high availability.	Backend / Database	Replication: at least 2 replicas	Ensures availability during maintenance or failure of primary database
DB-025	The system must implement automatic failover to a replica if the master node becomes unavailable.	Backend DB	Failover time $\leq$ 30 seconds	Reduces downtime and ensures service continuity
DB-026	Read replicas must be used for handling read-heavy workloads and reporting queries.	Backend / Database	At least 2 read replicas for load distribution	Improves performance and reduces load on the primary database
DB-027	Replication lag should not exceed 5 seconds between the master and replicas.	Backend / Database	Lag $\leq$ 5 seconds	Ensures near-real-time data consistency across replicas
DB-041	The system shall use asynchronous replication for reporting and analytics to prevent performance impact on the primary database.	Reporting / Analytics	Replication delay $\leq$ 10 seconds	Improves performance while maintaining data consistency in real-time
DB-042	Database failover must occur automatically without requiring manual intervention, ensuring zero downtime .	Backend / DB	Failover time $\leq$ 30 seconds	Ensures continuous service availability during outages

9.5. Compliance & Legal

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-028	All user and transaction data must be stored in compliance with GDPR for European users.	User Data / Transactions	GDPR-compliant data storage, encrypted data at rest	Required for compliance with EU data protection regulations
DB-029	Data retention policies must be implemented to ensure that transaction data is stored for a minimum of 7 years .	Transaction Data	Retention period: 7 years	Ensures compliance with financial industry regulations





DB-030	Database access must be logged and monitored to ensure that only authorized personnel access sensitive data.	Backend / Admin Panel	Logs stored for 1 year, access audit logging	Prevents unauthorized access and ensures accountability
DB-031	Data masking must be used for sensitive fields (e.g., user IDs, transaction details) in non-production environments.	Development / Testing	Masked fields for non-prod environments	Ensures compliance and privacy in non-production environments
DB-032	Database backups must comply with industry standards and ensure no unauthorized access to backup data.	Backup / Data Storage	Compliance with PCI-DSS, HIPAA, GDPR standards	
DB-043	System must ensure compliance with HIPAA for handling healthcare-related financial data .	Healthcare Data	HIPAA-compliant data storage and handling	Necessary for any medical-related data storage and transactions
DB-044	The database must allow legal document storage with digital signatures for transactions.	Legal Documents	Digital signature compliance	Complies with electronic document signing standards

9.6. Data Quality & Integrity

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-045	System must track and store all data modification timestamps for every field in the database.	Data Audit	Timestamp column on critical fields	Enhances auditing capabilities and ensures traceability of data changes
DB-046	Every database import process must include an error-handling mechanism to catch invalid records.	Data Import	Error logs, alert system for invalid records	Prevents bad data from being entered into the system

9.7. Performance & Scalability

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
DB-047	Database queries must be optimized using indexed fields for high-traffic queries.	Backend / User Queries	Indexing on user_id , transaction_id	Improves search performance and reduces query latency
DB-048	Query optimization must be done to ensure that all reports run in under 2 seconds for more than 95% of use cases.	Reporting, Data Retrieval	Report performance checks on query response time	Ensures optimal user experience when generating reports
DB-049	The database should handle 500,000+ concurrent connections without degradation in performance.	Backend / Database	Max concurrent connections: 500,000	Supports high-volume financial systems without performance degradation

9.8. Data Retention & Legal Compliance

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
----	-------------------------	------------	---------------------	-------------------





DB-050	System must store data related to user transactions and account activity for a minimum of 7 years for audit purposes.	Financial Records / User Data	Retention period: 7 years	Required for financial audit compliance and regulatory reporting
--------	---	-------------------------------	---------------------------	--

10. Server and Network Infrastructure

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
SN-001	System must be deployed using Kubernetes (K8s) for container orchestration and Docker for containerization.	Backend / Microservices	Kubernetes 1.27+ and Docker 24+	Provides scalable and automated deployment, enabling microservices architecture
SN-002	Virtual Private Cloud (VPC) must be configured to ensure that the database and internal services are isolated from the internet.	Cloud Infrastructure	Use AWS VPC or GCP VPC, with private subnets	Ensures secure communication and isolation of internal services
SN-003	The application must be hosted on cloud servers (e.g., AWS EC2, Google Compute Engine) with auto-scaling to handle traffic fluctuations.	Backend / Frontend	EC2 with auto-scaling, minimum instance type: t3.medium	Allows the system to scale dynamically with varying loads
SN-004	Load balancing using NGINX or AWS Elastic Load Balancer (ELB) must distribute traffic across application instances.	Frontend, API Gateway	Load balancing for web traffic and APIs	Ensures fault tolerance and improves the response time during peak traffic
SN-005	Services must be deployed with high availability (HA) across multiple availability zones (AZs) to minimize downtime.	All Services	Multi-AZ deployment on AWS or Google Cloud Platform	Increases availability and redundancy of the services
SN-006	The system must support redundant networking paths to prevent downtime in case of network failures.	Cloud Infrastructure	Dual internet connections, redundant routes via BGP	Prevents single points of failure in network connectivity
SN-007	All external-facing services (APIs, Web Frontend) must be served over HTTPS with a valid SSL/TLS certificate .	External APIs, Web Services	SSL/TLS certificates issued by trusted CA (e.g., Let's Encrypt)	Ensures secure communication between clients and services
SN-008	Firewall rules must be applied to restrict access to sensitive services and allow only authorized IPs to access backend systems.	Backend / API Gateway	Port restrictions, only allow specific IP ranges	Prevents unauthorized access and secures the internal network
SN-009	The platform must implement network monitoring using Prometheus and Grafana for real-time performance and health metrics.	Network Infrastructure	Metrics: CPU usage, memory, network throughput	Enables proactive monitoring of network health and traffic patterns





SN-010	Distributed Denial of Service (DDoS) protection mechanisms must be implemented via a Web Application Firewall (WAF) or Cloudflare .	Web Frontend, API Gateway	WAF protection for HTTP traffic, rate-limiting, IP blocking	Protects against DDoS and other malicious network attacks
SN-011	All sensitive data exchanged between services must be protected by encryption in transit, using TLS 1.2 or higher .	Backend / Frontend	TLS 1.2+, with certificates for internal services	Ensures data integrity and confidentiality across the network

10.1. Server Capacity & Performance

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
SN-012	Cloud instances must be provisioned with a minimum of 4 vCPUs and 16GB of RAM for production environments.	Production Environment	Minimum instance: 4 vCPUs, 16GB RAM	Ensures adequate capacity for production workloads
SN-013	Auto-scaling policies must be configured based on CPU usage > 70% and memory usage > 80% .	Cloud Infrastructure	Auto-scaling triggered at defined thresholds	Improves system efficiency during traffic spikes and avoids resource bottlenecks
SN-014	The system must support at least 10,000 concurrent users per node for high-traffic periods.	Backend / API Gateway	Capacity to support concurrent users without service degradation	Ensures system performance during peak traffic
SN-015	Database servers must be provisioned with a minimum of 8 vCPUs and 32GB RAM for production environments.	Database Servers	Minimum instance: 8 vCPUs, 32GB RAM	Ensures efficient processing and querying of large datasets
SN-016	Real-time analytics queries should return within 5 seconds for more than 95% of cases.	Analytics / Data Warehouse	Query optimization for high-volume reporting	Optimizes performance of the analytics layer for user queries

10.2. Scalability & Load Management

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
SN-017	The system shall use horizontal scaling for both microservices and databases to handle increased load.	Backend / Microservices	Horizontal scaling via Kubernetes	Ensures scalability by distributing load across multiple instances
SN-018	The API Gateway must support API rate limiting of 1000 requests per minute per IP .	API Gateway	Rate limiting: 1000 req/min per IP	Protects against API abuse and ensures fair distribution of resources
SN-020	Load balancing must be used to distribute incoming traffic across multiple nodes to ensure optimal response times.	Web Frontend, API Gateway	NGINX, AWS ALB for HTTP load balancing	Improves service performance by distributing load evenly across nodes



10.3. Monitoring & Logging

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
SN-022	Prometheus must be used for real-time system monitoring (CPU, memory, disk, network).	DevOps / Monitoring	Prometheus with Grafana dashboards for visualizing metrics	Provides real-time monitoring to detect issues early
SN-023	All logs should be captured using ELK (Elasticsearch, Logstash, Kibana) stack for centralized logging.	Backend / Logging	Use ELK stack for log aggregation and visualization	Facilitates troubleshooting and proactive monitoring
SN-024	Alerting must be configured for system anomalies, including high CPU usage or failed database queries.	Backend / Monitoring	Alerts via email, Slack, or internal dashboard	Ensures rapid response to system failures or performance issues
SN-025	Audit logs must track all changes to financial data and be stored for at least 12 months.	Admin Panel, Backend	Logs retained for 12 months	Complies with legal and regulatory requirements for data tracking

10.4. Security & Compliance

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
SN-026	Firewall must be configured to block all inbound traffic except from allowed IPs and ports.	Cloud Infrastructure	Only allow traffic on ports 80 (HTTP), 443 (HTTPS)	Prevents unauthorized access to sensitive resources
SN-027	Two-factor authentication (2FA) must be enabled for all admin users accessing the system.	Admin Panel / Backend	2FA required for all admins using TOTP	Increases system security for privileged access
SN-028	DDoS protection must be implemented via services such as Cloudflare or AWS Shield.	Web Frontend, API Gateway	Cloudflare or AWS Shield for DDoS mitigation	Protects the platform from large-scale network attacks

11. Minimum Requirements

This category outlines the **hardware, software, cloud infrastructure, and environmental specifications** needed to run the Techleaf platform. This will include **minimum server specifications, storage requirements, network bandwidth**, and other key technical considerations.

11.1. Minimum Hardware Requirements

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
MR-001	The development environment should run on 2 CPUs, 8GB RAM, and 100GB SSD .	Development Environment	Minimum specs for dev environment	Ensures a smooth development experience without over-allocating resources
MR-002	The test environment must run on 4 CPUs, 16GB RAM, and 200GB SSD .	Testing Environment	Minimum specs for testing environment	Provides sufficient resources for running test





				cases and simulating real-world usage
MR-003	The production environment should support at least 8 CPUs , 32GB RAM , and 500GB SSD for general use.	Production Environment	Minimum specs for production, scalable for peak usage	Ensures the system can handle live traffic and high-demand loads
MR-004	Database servers in production must have at least 8 CPUs and 64GB RAM to support large datasets.	Database Servers	Minimum database specifications for handling large volumes of transactions	Ensures smooth processing of financial transactions and analytics
MR-005	Backup storage should be at least 1TB SSD with RAID 1 configuration for redundancy.	Backup Infrastructure	Redundant storage with automatic failover	Ensures reliability and data recovery capabilities in case of storage failures

11.2. Minimum Software Requirements

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
MR-006	The backend must run on Python 3.11+ with FastAPI for API management.	Backend Core Services	Python 3.11+ and FastAPI	Ensures fast, modern, and asynchronous API handling
MR-007	The frontend must be built using React 18+ and TypeScript 4.9+ , with Tailwind CSS for styling.	Frontend	React 18+, TypeScript 4.9+, Tailwind CSS	Modern, maintainable frontend with responsive design
MR-008	PostgreSQL 15+ must be used as the primary database, with MongoDB 5+ for NoSQL data.	Database	PostgreSQL 15+ and MongoDB 5+	Supports both relational and unstructured data handling
MR-009	Docker 24+ must be used to containerize the backend and microservices, with Kubernetes 1.27+ for orchestration.	DevOps / Backend	Docker 24+, Kubernetes 1.27+	Ensures scalability, ease of deployment, and microservice architecture
MR-010	Nginx should be used as a reverse proxy for load balancing and API gateway.	API Gateway / Web Services	Nginx for load balancing and reverse proxy	Improves availability, scalability, and routing efficiency

11.3. Minimum Cloud Infrastructure Requirements

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
MR-011	The platform must be deployed using Amazon Web Services (AWS) or Google Cloud Platform (GCP) .	Cloud Infrastructure	AWS or GCP for cloud services	Provides scalable cloud resources with global availability
MR-012	The cloud environment must include virtual private cloud (VPC) for secure networking.	Cloud Networking	Private VPC with secure subnets	Ensures isolation and security for sensitive services
MR-013	The system must implement auto-scaling for both compute instances and storage during traffic spikes.	Cloud Infrastructure	Auto-scaling configuration for high availability	Enables the platform to scale dynamically with traffic demands





MR-014	Cloud storage (e.g., AWS S3, Google Cloud Storage) must be used to store media assets, backups, and logs .	Cloud Storage	AWS S3, GCP Storage for data and backups	Ensures scalability and durability of media and backup storage
--------	--	---------------	--	--

11.4. Minimum Network & Bandwidth Requirements

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
MR-016	The minimum network bandwidth for the production environment must be 10Gbps to handle high-volume traffic.	Network Infrastructure	Minimum bandwidth: 10Gbps	Ensures high availability and fast data transfer in production
MR-017	The system must use Virtual Private Network (VPN) for all internal communication between services.	Network Communication	VPN encryption for inter-service communication	Ensures data security between internal services
MR-018	All API traffic must be served over HTTPS with TLS 1.2 or higher to ensure data security.	API Gateway / Backend	SSL/TLS encryption for all API requests	Prevents eavesdropping and ensures secure data transmission

11.5. Minimum Security Requirements

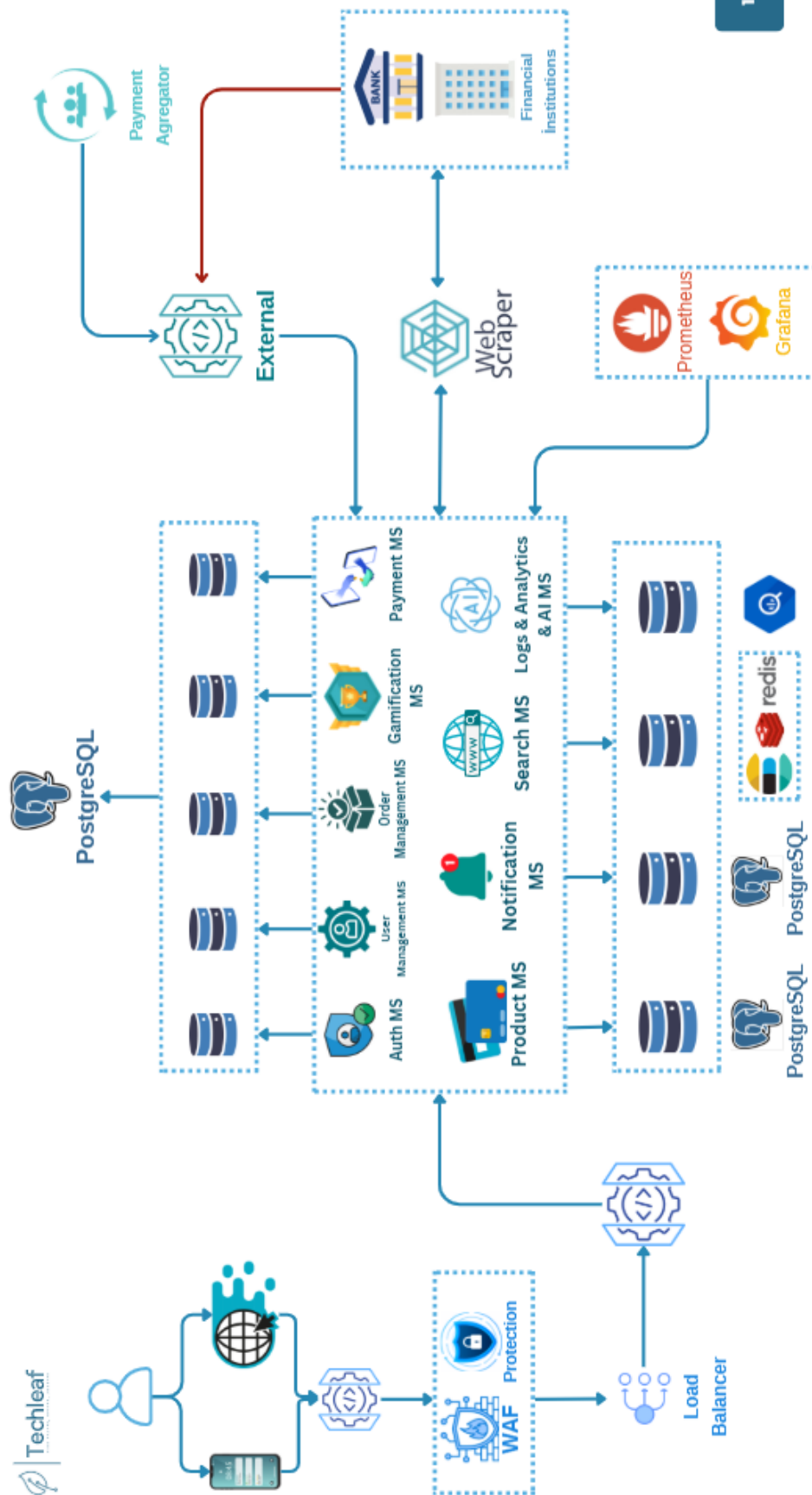
ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
MR-019	Two-Factor Authentication (2FA) must be enabled for all users accessing the admin panel.	Admin Panel, User Management	2FA with OTP, email, or app-based authentication	Enhances security for sensitive access
MR-020	Session management should expire after 15 minutes of inactivity, and require re-authentication.	User Sessions	15-minute inactivity session timeout	Prevents unauthorized access due to abandoned sessions
MR-021	All sensitive financial data (e.g., payment information) must be tokenized and stored according to PCI DSS standards.	Payments / Financial Data	Tokenized and PCI-DSS compliant	Complies with industry standards for handling payment data

11.6. Minimum Testing & Development Requirements

ID	Requirement Description	Applies To	Target / Constraint	Rationale / Notes
MR-022	Automated tests must be integrated into the CI/CD pipeline for all critical code changes.	Development Workflow	Coverage $\geq$ 80% for unit and integration tests	Ensures continuous code quality checks
MR-023	End-to-end (E2E) tests must be conducted for all new features before deployment to production.	Testing / QA	E2E test automation for all major features	Ensures the full application works as expected in real-world scenarios
MR-024	Static code analysis tools (e.g., SonarQube) must be used to ensure code quality before merging.	Development / CI	Code quality checks with SonarQube	Identifies code issues early and maintains quality standards



12. High-Level Architecture Diagram



15



13. Jira- MVP User Stories – Acceptance Criteria

Issue Type	Key	Summary	Status	Sprint
Epic	TZ-1	Product Search and Comparison	To Do	
Story	TZ-11	Compare Products Side-by-Side	To Do	TEC Sprint 1
Sub-task	TZ-71	Design the comparison page layout to display multiple products	To Do	TEC Sprint 1
Sub-task	TZ-72	Implement product data fetching for comparison	To Do	TEC Sprint 1
Sub-task	TZ-73	Ensure responsive design for mobile and desktop views	To Do	TEC Sprint 1
Sub-task	TZ-74	Write integration tests for product comparison logic	To Do	TEC Sprint 1
Story	TZ-12	Search Financial Products by Interest Rate	To Do	TEC Sprint 1
Story	TZ-13	Search for Financial Products by Category	To Do	TEC Sprint 1
Sub-task	TZ-68	Develop backend API to fetch financial products based on category	To Do	TEC Sprint 1
Sub-task	TZ-69	Create front-end search filters for categories.	To Do	TEC Sprint 1
Sub-task	TZ-70	Write unit tests for category filter logic	To Do	TEC Sprint 1
Story	TZ-14	Sort Products by Best Terms	To Do	
Story	TZ-15	Filter Products by Salary and Job Experience	To Do	
Story	TZ-16	Display Product Comparison with Key Metrics	To Do	
Story	TZ-17	Display Product Details on Click	To Do	
Story	TZ-18	Web Scraping for Loans	To Do	
Sub-task	TZ-99	Write unit tests for push notifications	To Do	
Sub-task	TZ-100	Develop scripts to scrape loan data (interest rates, loan amounts, fees)	To Do	
Sub-task	TZ-101	Ensure data consistency and accuracy for scraped loan products	To Do	
Sub-task	TZ-102	Schedule regular scraping jobs to update data	To Do	
Sub-task	TZ-103	Write unit tests for scraping logic	To Do	
Story	TZ-19	Web Scraping for Insurance Products	To Do	
Story	TZ-20	Web Scraping for Debit Cards	To Do	
Story	TZ-21	Web Scraping for Deposit Products	To Do	
Story	TZ-22	Web Scraping for Financial Product Comparison	To Do	
Epic	TZ-44	User Registration and Authentication	To Do	
Story	TZ-3	User Registration via Gmail	To Do	TEC Sprint 1
Story	TZ-4	User Login with Gmail	To Do	TEC Sprint 1
Story	TZ-5	User Login with Facebook	To Do	TEC Sprint 1
Story	TZ-6	User Login via Phone Number	To Do	TEC Sprint 1
Story	TZ-7	User Registration with Gmail Account	To Do	TEC Sprint 1
Story	TZ-8	User Registration with Phone Number (OTP Verification)	To Do	TEC Sprint 1
Story	TZ-9	Password Reset Functionality	To Do	
Story	TZ-10	Two-Factor Authentication Setup	To Do	
Epic	TZ-45	Loan Application Process	To Do	





Story	TZ-23	Fill Loan Application Form (Personal Details)	To Do	TEC Sprint 1
Story	TZ-24	Submit Loan Application	To Do	
Story	TZ-25	Select Loan Amount and Terms	To Do	
Story	TZ-26	Upload Required Documents (ID, Proof of Income)	To Do	
Story	TZ-27	Receive Personalized Loan Offers Based on Data	To Do	
Story	TZ-28	Select Loan Offer from Multiple Banks	To Do	
Story	TZ-29	Track Loan Application Status (Approved, Pending, Rejected)	To Do	
Epic	TZ-46	Bank Admin Panel	To Do	
Story	TZ-30	Bank Admin Login to Admin Panel	To Do	
Story	TZ-31	View Loan Applications in Admin Panel	To Do	
Story	TZ-32	Update Loan Application Status (Approved, In Progress, Rejected)	To Do	
Story	TZ-33	Generate Loan Application Reports	To Do	
Story	TZ-34	Assign Loan Officer to Loan Application	To Do	
Story	TZ-35	View Financial Product Data from Partner Banks	To Do	
Epic	TZ-47	Personalized Financial Assistance	To Do	
Epic	TZ-48	Notifications & Alerts	To Do	
Story	TZ-36	Send Email Notifications on Loan Status Updates	To Do	
Story	TZ-37	Send Email Notifications on Loan Status Updates	To Do	
Story	TZ-38	User Notification for New Loan Offers	To Do	
Story	TZ-39	Send Real-time Notifications for Status Change	To Do	
Epic	TZ-49	Chatbot Assistance	To Do	
Epic	TZ-50	User Data Privacy and Security	To Do	
Epic	TZ-51	User Role Management (For Banks)	To Do	
Epic	TZ-52	Platform Access for Unregistered Users	To Do	
Epic	TZ-53	System Monitoring and Analytics	To Do	
Epic	TZ-54	Search and Filtering	To Do	
Epic	TZ-55	Backend Integration and API	To Do	
Epic	TZ-56	Platform Performance and Scalability	To Do	
Epic	TZ-57	Real-Time Application Updates	To Do	
Epic	TZ-58	User Feedback and Reviews	To Do	
Epic	TZ-59	Gamification	To Do	
Epic	TZ-60	P2P Lending	To Do	
Epic	TZ-61	Financial Institutions Integration	To Do	





⚡ TZ-44 /

📌 TZ-3



User Registration via Gmail

+ Add ⚙ Apps

To Do ▾ ⚡

Field Tab Acceptance criteria

Acceptance Criteria

Given the user is on the registration page,
 When the user selects the "Sign up with Gmail" option,
 Then the system should redirect to the Gmail authentication page,
 And the user should be able to log in with their Gmail account,
 And the system should create a new account using the user's Gmail credentials.

Acceptance Criteria

Gherkin:

```
Given the user is on the login page, When the
user selects the "Login with Gmail" option, Then
the system should redirect to the Gmail login
page, And the user should be able to authenticate
and access their account.
```

Checklist:

- User can click "Login with Gmail".
- System redirects the user to the Gmail login page.
- User authenticates with Gmail credentials.
- User is logged in and redirected to the home page.

Lightshot



TZ-11

TZ-11



Field Tab Acceptance criteria

Acceptance Criteria

Gherkin:

Given the user has selected multiple financial products, When the user clicks the "Compare" button, Then the system should display a side-by-side comparison of the selected products, And the comparison should include key features such as interest rate, term length, and fees.

Checklist:

- User selects multiple products for comparison.
- User clicks the "Compare" button.
- System displays a side-by-side comparison of selected products.
- Comparison includes relevant features such as interest rate, term, fees, etc.

TZ-1 /

TZ-12



Search Financial Products by Interest Rate

+ Add

Apps

To Do ▾



Field Tab Acceptance criteria

Acceptance Criteria

Gherkin:

Given the user is on the financial product search page, When the user enters a desired interest rate range, Then the system should display all products within the specified range.

Checklist:

- User enters the desired interest rate range.
- System displays all matching financial products.
- The results are sorted by interest rate.

TZ-45 /

TZ-23



+ Add

Apps

To Do ▾



Field Tab Acceptance criteria

Acceptance Criteria

Gherkin:

Given the user is on the loan application page, When the user fills in their personal details in the form, Then the system should validate the form and allow the user to proceed to the next step.

Checklist:

- User can enter personal details (name, address, date of birth).
- System validates the form fields.
- User can proceed to the next step after validation.

TZ-1 /

TZ-13



Search for Financial Products by Category

+ Add

Apps

To Do ▾



Field Tab Acceptance criteria

Acceptance Criteria

Gherkin:

Given the user is on the financial product search page, When the user selects a product category, Then the system should display all products within the selected category.

Checklist:

- User selects a category (e.g., loan, credit card).
- System displays all products within that category.
- The results are relevant to the selected category.